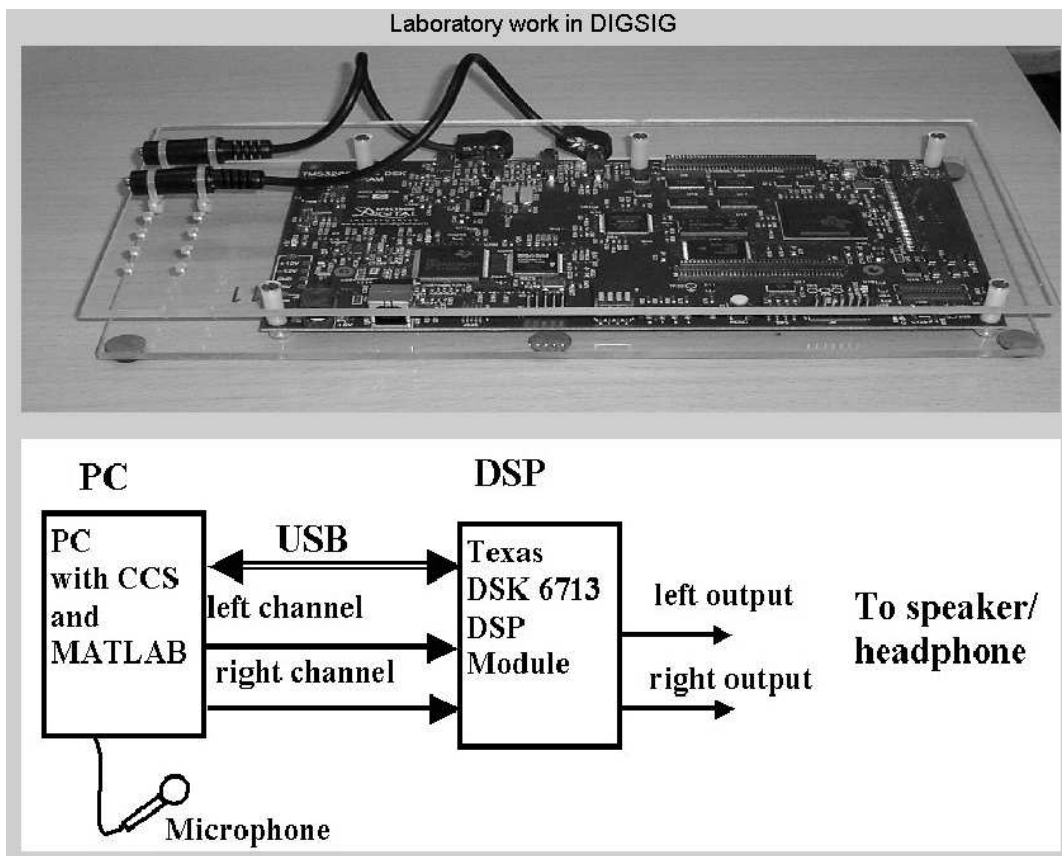




LUNDS TEKNISKA HÖGSKOLA
Lunds universitet

Digital Signalbehandling i Multimedia



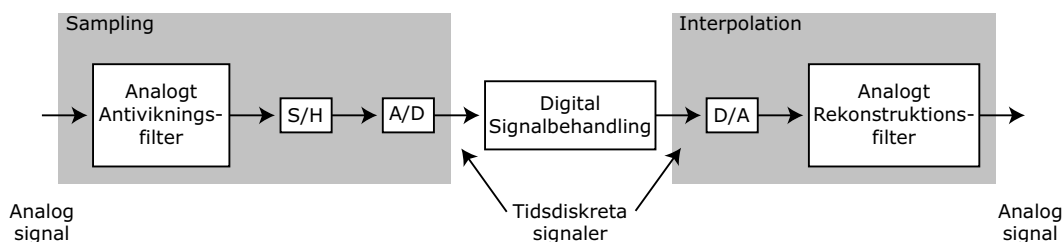
Laborationer med datorövningar
Institutionen för Elektrovetenskap
Lund 2006

Innehåll

1	Inledning till datorövningar	3
2	Övningsuppgifter för laborationerna	6
3	Matlab tips	10
4	Svar och lösningar till datoruppgifterna	11
5	Datorövning 1: Ljud och musik	14
5.1	Ett system för uppspelning/syntes/sändning: Exempel synthesizer	14
5.2	Några ljudsignaler	14
5.3	MATLAB-uppgifter	15
6	Datorövning 2: Hjärta och medicin	18
6.1	Ett system för inspelning/analys/mottagning: Exempel EKG-mätning	18
6.2	En EKG-signal	18
6.3	MATLAB-uppgifter	19
7	Datorövning 3: Filtrering av bilder	21
7.1	Datoruppgifter	21
7.2	Filtrering av bilder	21
7.3	Ett FIR-filter för kantextraktion	22
8	Datorövning 4: Filterdesign och digital audioteknik	23
8.1	Digital Audioteknik i realtid	25
9	Datorövning 5: Tal och röstanalys	30
9.1	Ett system för digital behandling av analog signal: Exempel Signalbehandling av talsignaler	30
9.2	Några talsignaler	30
9.3	MATLAB-uppgifter	30
10	Datorövning 6: IIR-filter	33
10.1	Inledning	33
10.2	Förberedelseuppgifter	34
10.3	Laborationsuppgifter	36
10.4	Programbeskrivningar	40
11	Datorövning 7: DFT i teori och praktik	45
11.1	DFT av sinus och cosinus	45
11.2	DFT och faltning	45
12	Extra övningsuppgifter	48
13	Svar och lösningar till extra övningsuppgifter	50

1 Inledning till datorövningar

Det traditionella sättet att motivera digital signalbehandling är att man vill behandla en analog signal digitalt dvs med en dator eller med någon form av digital hårdvara. Syftet med detta är att det idag går att behandla signaler digitalt snabbt och billigt, på liten kretsytta och med stor flexibilitet. En principskiss över hur detta fungerar visas i Figur. 1 där en verklig analog signal *samlas* till en tidsdiskret signal bestående av en följd av värden eller *sampel*. Hur många sampel som produceras per sekund i denna process avgörs av den valda *samlingsfrekvensen*. Den tidsdiskreta signalen kan nu manipuleras på olika sätt med digital signalbehandling innan det är dags att *interpola* tillbaka signalen till den analoga världen och skicka den vidare till något lämpligt analogt system t.ex. en högtalare eller en antenn.



Figur 1: Allmänt system en analog signal behandlas digitalt och sedan återförs till en analog signal.

Redan i ett tidigt skede av kursen är det viktigt att förstå principiellt vad de olika delarna i detta system har för uppgift. Därför följer nu en kort resumé:

Tidsdiskreta signaler har en mycket fundamental egenskap, nämligen att de inte vet hur snabbt de är inspelade eller hur snabbt de ska spelas upp. Tänk till exempel på en CD-skiva som innehåller en tidsdiskret signal. Denna signal vet inte själv vilket tidsintervall eller *samlingsintervall* det ska vara mellan sampel utan det är CD-spelaren (interpolatorn i bilden) som vet att sampel ska spelas upp med 44.1 kHz. Detta visste man även i studion där skivan spelades in (samlades). På grund av detta säger man ofta att en tidsdiskret signal har avståndet ett mellan sina sampel, dvs samlingsperioden 1 utan enhet (definition). Samplingsfrekvensen är inversen till samlingsperioden och denna blir därför också ett utan enhet. Alla frekvenser i den tidsdiskreta signalen anges nu i relation till samlingsfrekvensen som är ett. Dessa frekvenser kallas nu normerade frekvenser och ligger mellan 0 och 1. Det är först vid uppspelning med en lämplig samlingsfrekvens som dessa normerade frekvenser förvandlas till riktiga frekvenser eftersom det är då man bestämmer vad som ska stå på tidsaxeln.

En annan mycket viktig egenskap hos tidsdiskreta signaler är att den högsta frekvens som kan representeras av en tidsdiskret signal är den som har varannat sampel positivt och varannat negativt med samma amplitud. Denna frekvens har perioden 2 och därmed den normerade frekvensen 0.5. Man kan försöka spela in analoga signaler med snabbare variationer men dessa frekvenser kommer inte att kunna representeras korrekt av den tidsdiskreta signalen. Faktum är att alla frekvenser i den analoga signalen som ligger över den normerade frekvensen 0.5 motsvarande halva den riktiga samlingsfrekvensen kommer att se ut som lägre frekvenser i den tidsdiskreta signalen. Fenomenet kallas *vikning* och gör att den samplade signalen inte kan mappas tillbaka till den analoga världen eftersom de invikta högre frekvenserna nu tros vara de som syns inom det normerade frekvensintervallet 0 till 0.5. Som exempel kan ges två analoga toner på 100 och 350 Hz som samlas med 500 Hz. Tonen på 350 Hz ligger över halva samlingsfrekvensen och kan inte representeras av den tidsdiskreta signalen utan uppfattas istället uppfattas av den tidsdiskreta signalen som en ton mellan 0 och 250 Hz (här blir det 500-350=150 Hz). Återförs denna till den analoga världen kommer 100 Hz tonen

och den nya invikta tonen att spelas upp.

Lösningen på problemet är givetvis att bara tillåta inspelning av sådana toner som korrekt kan representeras av den tidsdiskreta signalen. Därför används ofta ett analogt *antivikningsfilter* som bara släpper igenom frekvenser under halva samplingsfrekvensen innan en signal överförs från den analoga till den digitala världen med hjälp av en sample-and-hold krets och en analog-till-digital-omvandlare. Nu är relationen mellan den analoga och den digitala världen entydig - det s.k. *samplingsteoremet* är uppfyllt.

Att samplingsteoremet är uppfyllt vid samplingen betyder också vi inte förlorar någon information i A/D-omvandlaren dvs att den ursprungliga analoga signalen kan rekonstrueras (interpoleras) exakt från den tidsdiskreta signalen. Detta går till så att en digital-till-analog-omvandlare håller varje sampels värde under en hel samplingsperiod på dess utgång. Resultatet blir en styckvis konstant signal med stegförändringar vid varje nytt samplingsintervall. Dessa stegförändringar mellan samplingsintervallen av frekvenser som är mycket högre än den högsta tillåtna frekvensen som är halva samplingsfrekvensen. I själva verket ligger hela skillnaden mot den ursprungliga signalen över halva samplingsfrekvensen. Därför används återigen ett analogt filter som här kallas ett *rekonstruktionsfilter* för att ta bort de frekvenser över halva samplingsfrekvensen som uppenbarligen inte skickades in i systemet. Nu ser den analoga utsignalen ut exakt som den analoga insignalen givet att vi samplade dubbelt så högt som den högsta förekommande frekvensen i insignalen. Om vi inte hade gjort detta, så hade antivikningsfiltret tagit bort de högsta frekvenserna i insignalen så att detta villkor uppfyllts och därmed undvikit att frekvenser hamnat på fel plats i frekvensskalan.

Slutligen några meningar om begreppet *filter*. Ett filter beskriver hur ett bestämt antal sampel från en insignal, fram till det senaste, ska viktas ihop för att åstadkomma en viss utsignal. Vikterna läggs i en följd och bildar en tidsdiskret sekvens - detta är filtrets *impulssvaret*. Filtrets impulssvaret är en representation av ett LTI system. En annan beskrivning av samma system är i form av en differensekvation. Det är mycket viktigt att inse att begreppen filter och impulssvar inte är så märkliga som de först kan verka. Tidsdiskreta signaler är mycket vanligt förekommande ett filter är inte något konstigare än ett verktyg för att kombinera indata för att åstadkomma önskad utsignal. Låt oss ta ett exempel:

Låt säga att du varje vecka sätter in $x[n]$ kronor på banken där n är numret på veckan sedan du började. Vi kallar $x[n]$ för en signal - en tidsdiskret insignal till ett system. Nu ska vi bestämma vad du vill ha för utsignal; detta bestämmer hur systemet ska se ut. Som utsignal till systemet väljer vi till exempel summan av dina insättningar den senaste månaden dvs under de senaste fyra veckorna. Differensekvationen till systemet beskriver då att utsignalen för nuvarande vecka är lika med summan av de fyra sista insättningarna där var och en viktas med 1, dvs: $y[n] = 1x[n] + 1x[n-1] + 1x[n-2] + 1x[n-3]$. Filtret viktat in de senaste 4 insättningarna och sägs därmed ha *längden* 4. Filtret eller impulssvaret är sekvensen av vikterna med början på den senaste dvs $[1 \ 1 \ 1 \ 1]$. Denna typ av filter har begränsat minne och viktat bara in ett begränsat antal sampel; filtret kallas därför för ett *FIR (Finite Impulse Response)* filter.

Nu säger vi istället att du vill att ditt system/filter ska beräkna ditt saldo. Ett enkelt sätt att beräkna saldot är att uppdatera saldot vid varje insättning dvs det nya saldot är det gamla saldot plus den nya insättningen. Differensekvationen för detta blir $y[n] = 1y[n-1] + 1x[n]$. Det som är viktigt att se här är att det nu inte bara är insignalen som ingår i beräkningen utan även gamla utsignaler. Detta komplicerar beräkningen av impulssvaret eftersom det per definition är hur *insignaler* ska viktas för att åstadkomma en viss utsignal. Om gamla utsignaler ingår i systemekvationen kallas systemet rekursivt. För att lösa problemet inser vi att samma ekvation kan användas för gamla utsignaler t.ex. $y[n-1] = 1y[n-2] + 1x[n-1]$ och $y[n-2] = 1y[n-3] + 1x[n-2]$ etc. Den viktiga insikten här är att utsignalen i systemet beror av alla insignaler (insättningar) fram till nu. Impulssvaret blir i detta fallet oändligt långt $[1 \ 1 \ 1 \ 1 \ \dots \ \dots]$ och kallas för ett *IIR (Infinite Impulse Response)* filter.

Nu har vi i detta enkla exempel bara använt vikterna 1 men med olika vikter på olika gamla insignaler visar sig filterstrukturen vara mycket kraftfull. Varje signal kan delas upp i de

frekvenser den består av och i fortsättningen av denna kurs ska vi se hur våra filter kan användas för att framhäva, dämpa eller fördröja olika delar av dessa frekvenser. I själva verket är det exakt samma sak att kombinera ihop insignaler vid olika tidpunkter till en utsignal som att förstärka, dämpa och fördröja en signalens olika frekvenskomponenter.

Översikt

Den detaljerade teoretiska förståelsen av de olika blocken i systemet ovan kommer att växa fram under kursens gång. Datorövningarna har tre syften: 1) att efterhand ge kött på benen kring hur de teoretiska verktygen kan användas, 2) att beskriva hur komplicerade system kan byggas upp kring de beskrivna komponenterna och 3) att ge exempel på verkliga signaler från tillämpningsområden där digital signalbehandling används. Praktisk utveckling inom signalbehandlingsområdet genomförs ofta i MATLAB-miljön. Därför syftar datorövningarna också till att ge MATLAB-erfarenhet och till att lära ut hur de olika signalbehandlingsverktyg som finns tillgängliga kan användas och tolkas. Steget från utveckling i MATLAB till en realtidsalgoritm i C, som kan köras i en mikroprocessor eller digital signalprocessor, går igenom i våra fortsättningskurser.

Vecka för vecka är fokus för datorövningarna följande:

- 1 Ljud och musik - ett system för uppspelning/sändning/syntes.
- 2 Hjärta och medicin - ett system för inspelning/mottagning/analys.
- 3 Filtrering av bilder.
- 4 Filter design och digital audioteknik (kräver speciell signalprocessor).
- 5 Tal och röstanalys - ett system för digital behandling av analog signal.
- 6 IIR-filter (kräver speciell signalprocessor).
- 7 DFT i teori och praktik.

2 Övningsuppgifter för laborationerna

Laboration vecka 1

Förberedelseuppgift: Läs igenom avsnittet om datorövning 1, Ljud och musik.

Laborationsuppgifter:

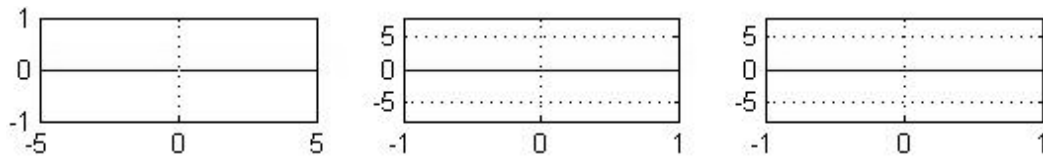
D1.1 Skissa först funktionerna nedan och plotta sedan upp dem i matlab. Gradera axlarna.
 $f_0 = 0.8$, $T_0 = 4$.

a) $x(n) = \sin(2\pi f_0 n) / 2\pi f_0 n \quad -5 < n < 5$

b) $X(f) = \sin(2\pi T_0 f) / \sin(2\pi 0.5 f) \quad -1 < f < 1$

c) $X(f) = |\sin(2\pi T_0 f) / \sin(2\pi 0.5 f)| \quad -1 < f < 1$

Fyll i signalerna i nedanstående figur.



D1.2 Bestäm grundton och vågform för ljudsignaler innehållande vokalljuden 'a' och 'e'. Sampla in med t ex wavestudio och spara signalerna som .wav-filer. Läs sedan in i matlab och lyssna. Välj ut ett kort avsnitt med ginwav. Bestäm grundtonens frekvens (stämbandens svängningsfrekvens) för bägge signalerna. Försök också förklara skillnaden i utseende mellan signalerna.

Svar: Grundtonerna är i), ii)

D1.3 Övningsuppgift 2.8 i läroboken. Nedanstående Matlab-kod är given. Teckna det matematiska uttrycket för funktionen $x(t)$ och skissa signalen. Kolla sen i Matlab att det blev rätt plott.

```
dt=1/100;  
tt=-1:dt:1;  
F0=2;  
xx=300*real(exp(j*2*pi*F0*(tt-0.75)));  
plot(tt,xx), grid,  
title('SECTION of a SINUSOID'),xlabel('TIME (sec)'),
```

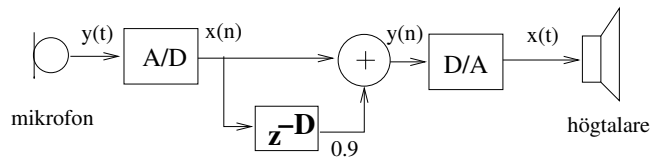
D1.4 Gå nu igenom det förberedda exemplet Ljud och musik (datorövning 1).

***D1.5** Rita upp följande sinussignaler i intervallet $0 < t < 20$ ms i $N = 100$ punkter och i $N = 10$ punkter. Plotta sedan i matlab. Använd kommandona stem och plot.

a) $x(t) = \sin(2\pi 200t)$

b) $x(t) = \sin(2\pi 800t)$

***D1.6** För att åstadkomma ekoeffekt använder vi oss av nedanstående koppling (z^{-D} fördröjer signalen D sampel).



Simulera detta i MATLAB och lyssna på effekten.

Exempel på MATLABkod:

```
t=-1:0.0001:1; %generera 2 s lång tidsvektor, FS=1/10000
x=vco(t,[300,800],10000); %generera en svept sinussignal mellan 300 och 800 Hz
soundsc(x,10000); %lyssna
noll=zeros(1,500); %generera 500 nollor
x1=[x,noll];x2=[noll,0.9*x];
y=x1+x2; %generera ekot
soundsc(y,10000); %lyssna
```

Välj nytt x (samplat tal och lyssna på ekoeffekten).

Laboration vecka 2

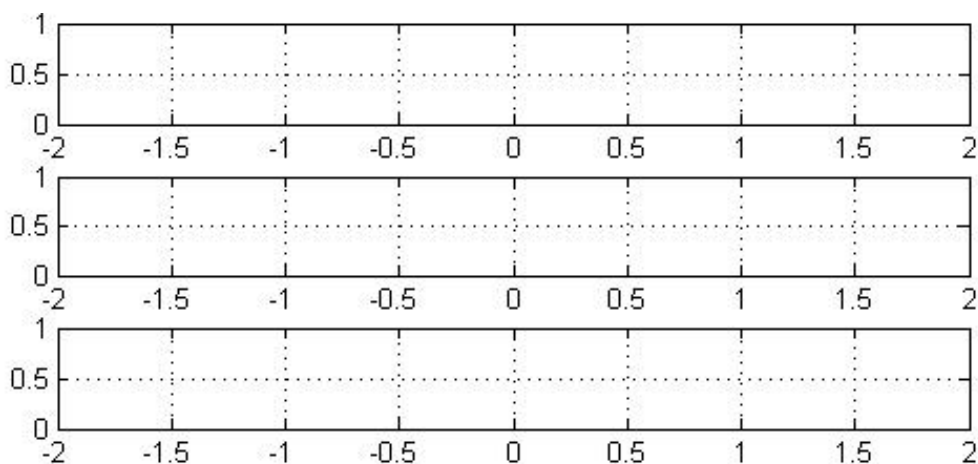
Förberedelseuppgift: Läs igenom avsnittet om datorövning 2, Hjärta och medicin.

Laborationsuppgifter:

D2.1 Vi vill plotta frekvensinnehållet i några digitala signaler och vi utnyttjar definitionen av fouriertransform (analog) där vi approximerar integral med summa. Skriv en rutin i Matlab som beräknar och plottar beloppet av fouriertransformen för $-2 < \hat{f} < 2$ ($\hat{f} = f/fs$).

- Använd rutinen för att plotta spektrum av $x(n) = \sin(2\pi 0.2 n)$.
- Använd rutinen för att plotta spektrum av $x(n) = \sin(2\pi 0.2 n)/(2\pi 0.2 n)$.
- Plotta spektra för vokalljuden 'a' och 'e'.

Fyll i spektra i nedanstående figur.



D2.2 Lös övningsuppgift 4.6 i läroboken.

D2.3 Gå sedan igenom det förberedda exemplet Hjärta och medicin (datorövning 2).

Laboration vecka 3

Förberedelseuppgift: Läs igenom avsnittet om datorövning 3, Filtrering av bilder.

Laborationsuppgifter:

D3.1 Beräkna amplitudspektrum $|H(\hat{f})|$ för ett filter med impulssvaret

a) $h(n) = \{1 \ 1 \ 1 \ 1\}$

b) $h(n) = \{0.25 \ 0.5 \ 0.25\}$

Plotta sedan i matlab och gradera axlarna.

D3.2 Lös övningsuppgift 5.12 i läroboken.

D3.3 Lös rötterna (nollställena) till nedanstående system i matlab. Svara med både i kartesiska och polära koordinater. Plotta sedan polnollställesdiagram i matlab (zplane).

a) $H(z) = 1 + z^{-1} + z^{-2} + z^{-3} + z^{-4}$

b) $H(z) = 1 + z^{-4}$

c) $H(z) = 1 - z^{-4}$

D3.4 Gå sedan igenom det förberedda exemplet Filtrering av bilder (datorövning 3).

Laboration vecka 4

Förberedelseuppgift: Läs igenom avsnittet om datorövning 4, Filterdesign och digital audioteknik. och gör de förberedelseuppgifter som där är beskrivna.

Laborationsuppgifter:

D4.1 Laboration: Filter design och digital audioteknik. Här utnyttjas ett speciellt DSP-kort som finns i datorerna i labsalen. Se handledning datorövning 4.

Laboration vecka 5

Förberedelseuppgift: Läs igenom avsnittet om datorövning 5, Tal och röstanalys.

Laborationsuppgifter:

5.1 Lös övningsuppgift 6.4, i läroboken.

5.2 Lös övningsuppgift 7.4 i läroboken.

5.3 Lös övningsuppgift 7.7 i läroboken.

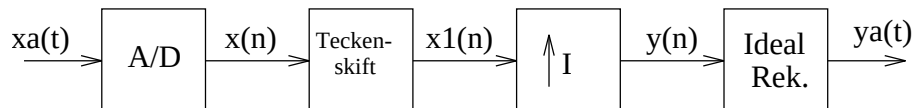
D5.4 Gå sedan igenom det förberedda exemplet Talkodning och röststyrning (datorövning 5).

***D5.5** Bestäm 3-dB bandbredden för filtren ($0 < \alpha < 1$).

$$H_1(z) = \frac{1-a}{1-az^{-1}} \quad H_2(z) = \frac{1-a}{2} \frac{1+z^{-1}}{1-az^{-1}}$$

Plotta amplitudfunktionerna i matlab för $a = 0.5$ och uppskatta därur bandbredderna.

***D5.6** Bestäm i Matlab utsignalen från nedanstående krets då insignalen är $x_a(t) = 2 \cdot \cos(2\pi \cdot F_0 t)$ då $F_0 = 6000\text{Hz}$ och $F_s = 10000\text{Hz}$.



Teckenskiiftaren utför operationen

$$x_1(n) = (-1)^n x(n)$$

och uppsamplaren

$$y(n) = \begin{cases} x_1\left(\frac{n}{2}\right) & n \text{ jämn} \\ 0 & n \text{ udda} \end{cases}$$

Rekonstruktionen är ideal och sker med samplingstakten $2F_s$.

Laboration vecka 6

Förberedelseuppgift: Läs igenom avsnittet om datorövning 6, IIR-filer och gör de förberedelseuppgifter som där är beskrivna.

Laborationsuppgifter:

D6.1 Laboration: IIR-filer. Se handledning datorövning 6.

Laboration vecka 7

Förberedelseuppgift: Läs igenom avsnittet om datorövning 7, DFT i teori och praktik..

Laborationsuppgifter:

D7.1 Låt $X(k)$ vara 8 punkters DFT av $x(n)$ med $x(n) = \{0, 1, 1, 3, 8, 7, 2, 2\}$

a) Beräkna $y(n) = IDFT(X^*(k))$.

b) Beräkna $y(n) = IDFT((-1)^k X(k))$.

Kolla räkningarna i matlab med fft, ifft. Låt $x(n)$ vara en ljudsignal och lyssna på $y(n)$ i a) respektive b).

D7.2 Lös övningsuppgift 8.16 i läroboken.

D7.3 Gå sedan igenom det förberedda exemplet DFT i teori och praktik (datorövning 7).

3 Matlab tips

Använbara Matlabkommando

Nedan ges några Matlabkommando som vi flitigt kommer att använda (utnyttja matlabs help för att få mer information).

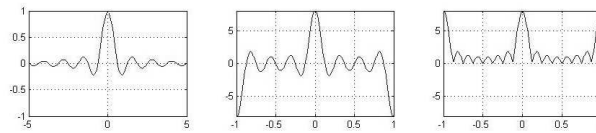
```
xin=wavread('signal.wav'); läser in signal i Matlab från .wav file  
X=fft(x,N); Beräknar fft(x) i N punkter  
plot(abs(X)); plottar beloppet av X;  
specgram(x); plottar spektrum av x som funktion av tiden  
Z=roots(B); löser rötter till polynom  
B=poly(Z); beräknar polynom från rötterna  
subplot(211),plot(x) plottar x i övre del av plotfönstret
```

Egna matlabrutiner som finns för nedladdning

```
x=ginwav(xx); Väljer ut del av signal med hjälp av mus-  
sigfftp(x,Fs,Nx,Fmax); Plottar spektrum av x med gradera x-axel.  
dspfour(x); Plottar spektrum över valfritt frekvensintervall.
```

4 Svar och lösningar till datoruppgifterna

D1.1 (se boken ex 2.8)



D1.2 Beror på er insignal.

D1.3 $xx(t) = 300 \cos(2\pi 2t - \pi)$

D2.1 Exempel på Matlabkod

```
function [X,f]=dspfour(x,Fs,Nf,fmax)
if nargin < 2 Fs=1; end; if nargin < 3 Nf=500; end;
if nargin < 4 fmax=2*Fs; end;
Nt=length(x);
if (Nt > 500) Nt=500; disp(' Warning: Length of x reduced to 500 sample values'); end
if (Nf > 500) Nf=500; disp(' Warning: Length of X reduced to 500 frequency values'); end
f=zeros(Nf,1); X=zeros(Nf,1);
for kf=1:Nf
f(kf)=(2*fmax/Fs*(kf-1)/Nf-fmax/Fs);
X(kf)=0; for kn=1:Nt X(kf)=X(kf)+x(kn)*exp(-j*2*pi*f(kf)*kn);
end;
end;
subplot(311),plot(x(1:Nt)); subplot(312),plot(Fs*f,abs(X));
```

D2.2 Exempel 4.6c i boken $f_s > 24 \text{ kHz}$

D3.1 a) $H(\hat{f}) = \frac{\sin(2\pi \hat{f} 2)}{\sin(2\pi \hat{f} / 2)} e^{-j2\pi \hat{f} 3/2}$

b) $H(\hat{f}) = (0.5 + 0.5 \cos(2\pi \hat{f})) e^{-j2\pi \hat{f}}$

Exempel på Matlabkod för a)

```
N=100;f=0:1/N:1-1/N; H=abs(sin(2*pi*f*2)./sin(2*pi*f*0.5)); plot(f,H);
eller med fft
N=100;h=[1,1,1,1];H=fft(h,N);plot([0:1/N:1-1/N],abs(H));
```

D3.2 Exempel 5.12 i boken $h(n) = [2, 2, 2, 2, 2]$

D3.3 a) $Z = \text{roots}([1, 1, 1, 1, 1]); P = \text{roots}([1, 0, 0, 0, 0]); \text{zplane}(Z, P);$

b) $Z = \text{roots}([1, 0, 0, 0, 1]); P = \text{roots}([1, 0, 0, 0, 0]); \text{zplane}(Z, P);$

c) $Z = \text{roots}([1, 0, 0, 0, -1]); P = \text{roots}([1, 0, 0, 0, 0]); \text{zplane}(Z, P);$

D5.1 Exempel 6.4 i boken $H(\hat{\omega}) = (-3 + 4 \cos(\hat{\omega})) e^{-j \hat{\omega}}$

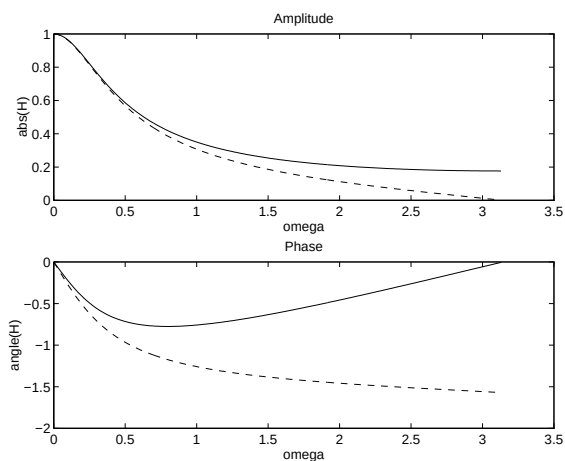
D5.2 Exempel 7.4 i boken $H(\hat{\omega}) = 0.33 (1 + 2 \cos(\hat{\omega})) e^{-j \hat{\omega}}$

D5.3 Exempel 7.7 i boken $Z = \text{roots}([1, .5, .5, 1]), P = \text{roots}([1, 0, 0, 0]); \text{zplane}(Z, P);$

D5.5

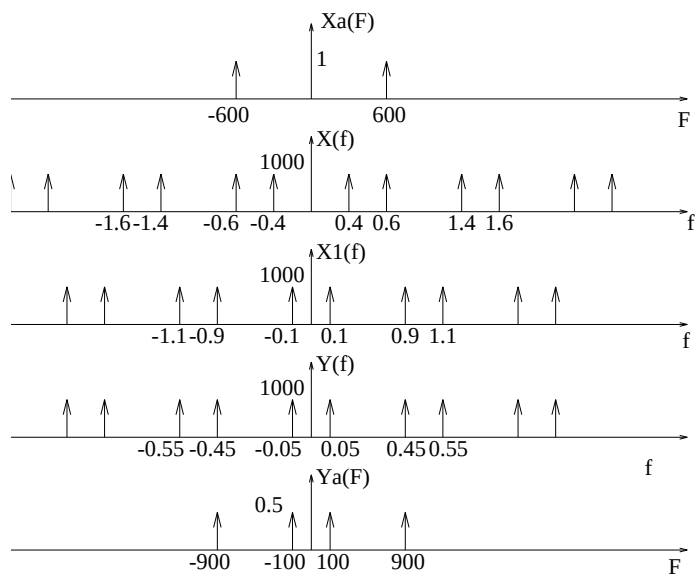
$$H_1(\omega) = \frac{1-a}{1-ae^{-j\omega}} \quad |H_1(0)| = \frac{1-a}{1-a} = 1$$

$$\begin{aligned}
|H_1(\omega_{3dB})| &= |H(0)| \cdot \frac{1}{\sqrt{2}} = \frac{1}{\sqrt{2}} \\
\Rightarrow |H_1(\omega_{3dB})|^2 &= \frac{(1-a)^2}{|1 - a \cos \omega_{3dB} + ja \sin \omega_{3dB}|^2} = \frac{1}{2} \\
\Rightarrow \omega_{3dB} &= \arccos \frac{4a - a^2 - 1}{2a} \\
H_2(\omega) &= \frac{1-a}{2} \frac{1+e^{-j\omega}}{1-ae^{-j\omega}} |H_2(0)| = \frac{1-a}{2} \frac{2}{1-a} = 1 \\
\Rightarrow |H_2(\omega)|^2 &= \left(\frac{1-a}{2}\right)^2 \frac{|1+\cos\omega - j\sin\omega|^2}{|1-a\cos\omega + aj\sin\omega|^2} = \frac{1}{2} \\
\Rightarrow \omega &= \arccos \frac{2a}{1+a^2}
\end{aligned}$$



H_2 är bäst ty nollställe i $z = -1$.

D5.6



$$y_a(t) = \cos(2\pi 100t) + \cos(2\pi 900t)$$

Exempel på Matlabkod:

```
F0=6000;Fs=10000;t=0:1/Fs:2; x=2*cos(2*pi*F0*t); soundsc(x,Fs);
plot(abs(fft(x)));
x1=(-1).^n*x;soundsc(x1,Fs);plot(abs(fft(x1)));
y=zeros(1,2*length(x1));y[1:2:2*length(x1)]=x1;
soundsc(y,2*Fs);plot(abs(fft(y)));
```

Gör gärna om med ljud som insignal.

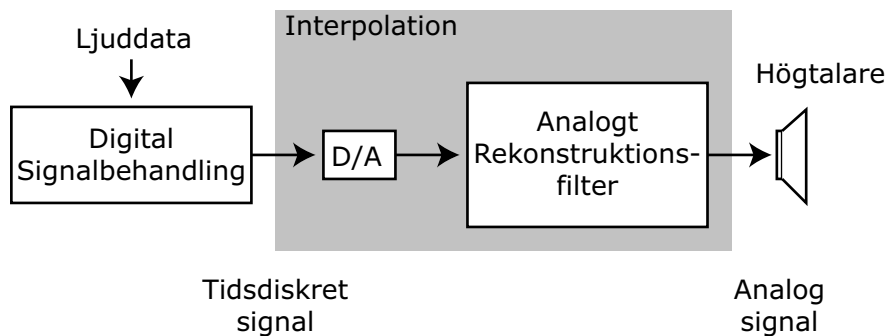
- D7.1** a) $y(n) = x(-n) = \{0, 2, 2, 7, 8, 3, 1, 1, \}$.
 b) $y(n) = x(n - 4, \text{modulo } 8) = \{8, 7, 2, 2, 0, 1, 1, 3\}$.

D7.2 Exempel 8.16 i boken: 1C, 2A, 3E, 4saknas, 5B, 6D

5 Datorövning 1: Ljud och musik

5.1 Ett system för uppspelning/syntes/sändning: Exempel synthesizer

Det första systemet vi ska titta närmare på visas i figur 2 (systemet motsvarar högra halvan i figur 1) och beskriver hur digitala data omvandlas till en analog signal. Detta system



Figur 2: System för uppspelning/syntes/sändning: Exempel synthesizer.

är mycket användbart och används t.ex. för uppspelning av lagrade digitala ljuddata i en CD-spelare, för att skapa ljud och musik digitalt i en synthesizer och för att sända ut data på om man ersätter högtalaren i bilden med en antenn. Vi kallar därför detta system för ett uppspelnings/syntes/sändningssystem. Vi repeterar funktionen för de olika blocken: Ljud- eller datasampelen som ska spelas upp eller sändas iväg finns i en tidsdiskret (och även kvantiserad) signal. Varje sampel i denna signal förvandlas i D/A-omvandlaren till någon sorts puls (vanligen rektangelformad med längden en samplingsperiod). Nu har vi en analog signal men i skarvarna mellan de skapade pulserna finns skarpa kanter. Dessa vassa kanter innehåller höga frekvenser och övertoner (enligt Fouriertransformen: ju skarpare kanter i en signal desto fler övertoner måste den byggas upp av). Eftersom vi vet att dessa höga frekvenser (högre än halva samplingsfrekvensen) inte fanns med i den ursprungliga signalen tas de bort i ett rekonstruktionsfilter. I ljudfallet måste de skarpa kanterna (de höga frekvenserna) bort för att ljudet ska låta som det ska och i sändarsystem bör de tas bort eftersom de påverkar ett mycket stort frekvensområde. Det normala är att man bara får utnyttja en viss bandbredd och då kan man inte skicka ut en massa skarpa kanter". I den här datorövningen ska du öka din förståelse kring tidsdiskreta signaler och bli säker på relationen mellan samplingsfrekvens och normerad frekvens. Teorin illustreras här med hjälp av ljud och din uppgift är att i MATLAB skapa en enkel synthesizer, dvs ett system som skapar toner av olika ljud och kombinerar dessa till musik. Den digitala signalbehandlingen utförs av datorn och ljudkortet, som innehåller just en D/A-omvandlare och ett rekonstruktionsfilter, används för interpolation innan den analoga signalen skickas till högtalaren.

5.2 Några ljudsignaler

Vi bestämmer oss för att använda CD-kvalitet på våra ljud, dvs de ska spelas upp med samplingsfrekvensen 44,1 kHz (sampler/sekund). Vidare bestämmer vi oss för ett ganska snabbt tempo på 1 takt/s i vår låt. Detta betyder att vi får 44100 sampler/takt. En takt består av 4 slag (s.k fjärdedelar) och varje slag blir då 11025 sampler långt. På varje fjärdedel går det två åttondedelar dvs hälften så korta toner. Dessa får då längden 5512.5 sampler. Eftersom sampler är heltal måste vi om vi ska använda åttondedelar göra varannan till längden 5512 och varannan till längden 5513.

Till denna övning behövs filerna: piano1.mat, shout1.mat, drum1.mat och hi1.mat. Dessa

filer innehåller ljud som är samplat med 44,1 kHz. Om Matlabfilerna inte finns på den dator du använder kan de laddas ner från kursens hemsida.

5.3 MATLAB-uppgifter

Uppgift 1: Ladda ljudfilerna genom att skriva

```
load piano1; load shout1; load drum1; load hi1;
```

Uppgift 2: Du ska nu undersöka dina inlästa variabler på tre sätt: 1) Skriv

```
size(piano1),
```

MATLAB svarar nu med två värden. Det första anger hur många rader ljuddatamatrisen innehåller och det andra anger hur många kolonner matrisen innehåller. Slutsatsen här blir att piano1 innehåller 5513 sampel staplade på höjden. Testa de andra ljuden på samma sätt. 2) Lyssna på ljuden dvs skicka ljuddata genom interpolatorn i ljudkortet till högtalaren. Som har nämnts tidigare innehåller en tidsdiskret signal ingen information om dess verkliga samplingsperiod. Detta måste vi tala om för ljudkortet som spelar upp ljudet därefter, i detta fallet 44100 Hz. Skriv

```
FT=44100;
soundsc(piano1,FT);
soundsc(shout1,FT);
soundsc(drum1,FT);
soundsc(hi1,FT);
```

Slutligen, 3) kan du titta på hur ljudsignalerna ser ut genom att skriva

```
figure; stem(drum1);
```

Om du istället för att rita en stapel för varje sampel vill binda ihop topparna på staplarna dvs rita en sammanhängande signal kan du istället skriva

```
figure; plot(drum1);
```

Uppgift 3: Nu ska vi skapa en liten bakgrundslåda för vår låt. Vi väljer att använda fyra takter där en ton ska ljuda under var och en av takterna. Vi väljer tonerna C4=261.63 Hz, A3=220 Hz, F3=174.61 Hz och G3=196 Hz. För in i MATLAB

```
C4=261.63; A3=220; F3=174.61; G3=196;
```

Vi väljer att använda en ren sinuston för detta ljudet. När vi skapar den tidsdiskreta signalen måste vi ange den normerade frekvensen eftersom signalen själv inte vet hur snabbt den ska spelas upp. Vi vill att den första tonen ska ljuda med 261.63 Hz om vi spelar upp med 44,1 KHz. Detta blir fallet om vi använder den normerade frekvensen 261.63/44100 dvs C4/FT. I MATLAB skapar vi först en vektor som numrerar samplen under en takt. En takt innehöll 44100 sampel och vektorn går därför mellan 0 och 44099. Skriv i MATLAB

```
Ntakt=44100; n=0:Ntakt-1;
```

Vektorn n ligger på längden (kan kollas med `size`) och måste transponeras till att ligga på höjden för att matcha ljuddata. Detta görs med

```
n=n';
```

Tonerna skapas nu enligt

```
ton1=sin(2*pi*C4/FT*n); ton2=sin(2*pi*A3/FT*n);  
ton3=sin(2*pi*F3/FT*n); ton4=sin(2*pi*G3/FT*n);
```

Dessa kan nu läggas i en följd för att lyssnas på

```
bakgrundsslinga=[ton1;ton2;ton3;ton4];  
soundsc(bakgrundsslinga,44100);
```

Uppgift 4: Nästa steg är att lägga på trummor. De trumfiler som lyssnades på tidigare hade båda längden 11025 dvs en fjärdedels takt. Vi skapar ett enkelt komp i fyra takter.

```
komptakt=[drum1;hi1;hi1;hi1];  
komp=[komptakt;komptakt;komptakt;komptakt];  
soundsc(komp,44100);
```

Tillsammans med bakgrundsslingan blir detta (amplituden halverad för bakgrundsslingan)

```
soundsc(0.5*bakgrundsslinga+komp,44100);
```

Uppgift 5: Det som gjorde det enkelt att lägga till trumsamplingarna i förra uppgiften är att trummor inte har någon speciell frekvens och därmed inte behöver anpassas till vilket accord/vilken ton som spelas. Nu ska vi lägga till 8 åttondedelar piano på den korrekta tonen (samma som bakgrundsslingan) för varje takt. Problemet är att vi har en tidstidskret signal (piano1) med den normerade frekvensen 261,63/44100 dvs som ljuder med 261,63 Hz om den spelas upp med 44100 Hz. Ett sätt att få denna ton att låta lägre är att spela upp den långsammare men eftersom vi vill spela upp hela vårt musikstycke med 44100 Hz går inte detta. Vad vi helt enkelt måste göra är att ändra den normerade frekvensen i piano1 så att de olika tonerna kan spelas. Följande relationer blir nu intressanta: $C4/A3=6/5$, $C4/F3=6/4$ och $C4/G3=4/3$. Vi kan nu sampla ompiano1 så att den får de andra normerade frekvenserna med hjälp av dessa relationer och MATLAB-kommandot **resample** (kräver signal processing toolbox).

```
piano2=resample(piano1,6,5);  
piano3=resample(piano1,6,4);  
piano4=resample(piano1,4,3);
```

Kolla nu storleken på dessa variabler med **size**. Det som har hänt är att har dragits ut och blivit längre för att låta långsammare. Vi räknar med att denna påverkan är ganska liten och använder de första 5512/5513 (varannan) samplen till våra åttondedelar.

```
pianoC4=[piano1(1:5512);piano1(1:5513);piano1(1:5512);piano1(1:5513);  
         piano1(1:5512);piano1(1:5513);piano1(1:5512);piano1(1:5513)];  
pianoA3=[piano2(1:5512);piano2(1:5513);piano2(1:5512);piano2(1:5513);  
         piano2(1:5512);piano2(1:5513);piano2(1:5512);piano2(1:5513)];  
pianoF3=[piano3(1:5512);piano3(1:5513);piano3(1:5512);piano3(1:5513);  
         piano3(1:5512);piano3(1:5513);piano3(1:5512);piano3(1:5513)];  
pianoG3=[piano4(1:5512);piano4(1:5513);piano4(1:5512);piano4(1:5513);  
         piano4(1:5512);piano4(1:5513);piano4(1:5512);piano4(1:5513)];  
pianoslinga=[pianoC4;pianoA3;pianoF3;pianoG3];  
soundsc(pianoslinga,44100);
```

Tillsammans låter det nu

```
soundsc(0.5*bakgrundsslinga+komp+pianoslinga,44100);
```


Uppgift 6: Sång lagras numera ofta också som samplat ljud. Som ett sista steg lägger vi därför nu på ett lite skrik i slutet av varannan takt. Vektorn `shout1` har längden 44100 dvs en takt så vi lägger in 2 nolltakter mellan våra skriktakter med samma storlek som våra skriktakter

```
shoutslinga=[zeros(size(shout1));shout1;zeros(size(shout1));shout1];  
soundsc(shoutslinga,44100);
```

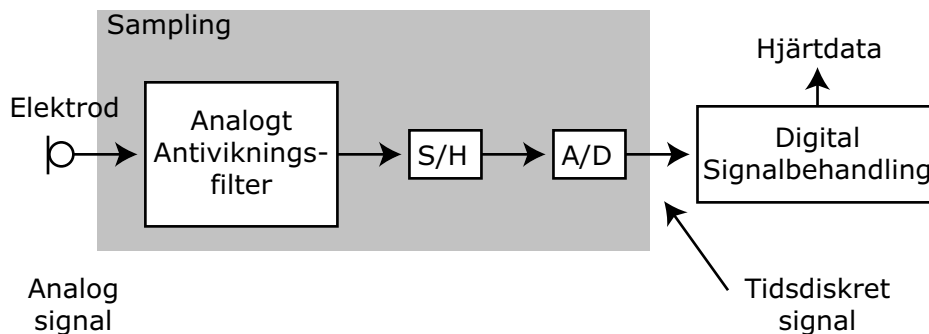
Hela resultatet bestående av egenskapade ljud, rena samplingar, frekvensomsamplade signaler och sång låter nu

```
soundsc(0.5*bakgrundsslinga+komp+pianoslinga+shoutslinga,44100);
```

6 Datorövning 2: Hjärta och medicin

6.1 Ett system för inspelning/analys/mottagning: Exempel EKG-mätning

Det andra systemet vi ska titta på visas i figur 3 (systemet motsvarar vänstra halvan i figur 1) och beskriver hur en uppmätt analog signal omvandlas till digital data som sedan processas i syfte att t.ex. ta fram information ur signalen.



Figur 3: System för inspelning/analys/mottagning: Exempel EKG-mätning.

Detta system är också mycket vanligt förekommande och används t.ex. för inspelning av ljud för digital lagring, för att spela in andra typer av signaler t.ex. medicinska (elektriska) eller mekaniska för analys och för att ta emot någon utsänd data om man ersätter mikrofonen i bilden med en antenn. Vi kallar därför detta system för ett inspelnings/analys/mottagnings-system. Vi repeterar funktionen för de olika blocken: En analog signal mäts med hjälp av en mikrofon, elektrod, antenn eller givare av annat slag. Denna signal har ett frekvensinnehåll med frekvenser upp till en viss högsta frekvens. Samplingsteoremet säger nu att vi måste sampla (A/D-omvandla) med en frekvens dubbelt så hög som denna frekvens för att få en korrekt tidsdiskret beskrivning av den analoga signalen. Om vi inte kan eller vill sampla så högt så kan vi sampla lägre förutsatt att den intressanta informationen ligger tillräckligt lågt i frekvens men då måste vi först filtrera bort frekvenser över halva samplingsfrekvensen så att dessa inte viks in på någon annan frekvens och förstör signalen. Detta görs med antivikningsfiltret. Vanligtvis får man in brus vid mätningar och detta brus kan ligga på höga frekvenser och därför använder man i princip alltid ett antivikningsfilter för A/D-omvandling. Sedan låses signalen under en samplingsperiod i S/H-kretsen så att A/D-omvandlaren får tid att läsa av värdet till varje sampel.

I den här datorövningen ska du öka din förståelse kring en signals frekvensrepresentation och hur DFT-verktyget kan användas. Teorin illustreras här med hjälp av en EKG-signal och din uppgift är att i MATLAB analysera denna och tala om vad de olika frekvenserna du ser betyder. Den analoga (elektriska) signalen är inspelad (samplad) på Kardiologen på lasarettet här i Lund och sedan digitalt lagrad. Den digitala signalbehandlingen utförs nu i efterhand av er dator (analysen gjordes också i realtid i datorn på lasarettet för att datorn skulle kunna larma om något inträffat).

6.2 En EKG-signal

Den inspelade EKG-signalen är samplad med 1 kHz dvs alla frekvenser över 500 Hz är bortfiltrerade. Detta är inget problem eftersom kroppen inte förväntas producera några snabbare förändringar än att de kan representeras av frekvenser upp till ca 250 Hz. Till denna övning behövs filen: ekg1.mat. Om Matlabfilen inte finns på den dator du använder kan den laddas ner från kursens hemsida.

6.3 MATLAB-uppgifter

Uppgift 1: Ladda EKG-filen genom att skriva

```
load ekg1;
```

Uppgift 2: Undersök den inlästa filen genom att skriva

```
size(ekg1),
```

Du fick nu reda på att ekg1-filen består av en tidsdiskret signal av längden 10000 sampel som är lagrad på bredden (1 rad och 10000 kolonner). Med samplingsfrekvensen 1000 Hz betyder detta en längd på 10 sekunder. Undersök filen vidare genom att skriva

```
figure; plot(ekg1);
```

Det du ser här är alltså en 10 sekunders EKG-signal innehållande 10 hjärtslag där de stora, smala spikarna motsvarar den elektriska impuls som drar ihop hjärtmuskeln och kramar ut blodet i kroppen och de små följande dipparna motsvarar återgången till ursprungsläget. Du ser också en långsam rörelse i baslinjen, dvs amplitudnivån mellan slagen. Denna variation orsakas dels av andning och dels av att nollreferensen kan variera lite vid mätning på kroppen. På y-axeln är det mikroVolt som är enheten.

Uppgift 3: Som nämnts många gånger så vet inte signalen hur snabbt den är inspelad. Därför använder MATLAB sampelnumreringen vid plottning som i föregående uppgift. Det är alltid mer intressant att ha en tidsskala på x-axeln dvs mappa över sampelnumreringen till tidpunkter. Detta kan göras baserat på samplingsfrekvensen (som vi måste känna till) genom att skriva

```
FT=1000; N=10000; n=0:N-1; t=n/FT;
```

där FT är samplingsfrekvensen, N är signalens längd, n är sampelindex räknat från noll och t är en ny tidsindexering till de 10000 samplen som ges av totala antalet sampel dividerat med antalet sampel per sekund vilket blir en skala i sekunder. Plotta signalen på nytt men nu genom att skicka med vilka värden som ska stå på x-axeln för varje sampel

```
plot(t,ekg1), xlabel('tid (s)'), ylabel('amplitud (mikroV)'),
```

Du ser nu redan att det verkar som pulsen ligger kring 1/sekund dvs 60 slag/minut.

Uppgift 4: Vi ska nu använda DFT-verktyget för att se vilka frekvenser vår signal innehåller; pulsen bör vara den tydligaste. I MATLAB beräknas DFT mha av kommandot **FFT**. FFT-kommandot vill veta i hur många jämnt fördelade frekvenser (punkter) den ska dela upp det normerade frekvensintervallet 0-1 i och beräkna spektrat i. Om man till exempel väljer att beräkna FFT för M punkter så kommer FFT att beräkna spektrat i punkterna (dvs de normerade frekvenserna) $\left[\frac{0}{M} \quad \frac{1}{M} \quad \frac{2}{M} \quad \dots \quad \frac{M-1}{M} \right]$. Om man vill veta vilka riktiga frekvenser detta motsvarar måste man multiplicera med samplingsfrekvensen. Vi väljer att beräkna FFT:n med $M = 10000$ punkter.

```
M=10000; ekgpek=fft(ekg1,M);
```

Vektorn ekgpek innehåller både amplitudfunktion och fasfunktion och är därför komplex. Vi är här bara intresserade av vilka frekvenser som förekommer och plottar därför bara amplitudfunktionen (`abs(ekgpek)`).

```
figure, plot(abs(ekgpek)),
```

Eftersom vi inte angav någon x-axeldata plottades spektrat bara mot numreringen av dess sampel. Denna information är svår tolkad, men vi vet ju vilka normerade frekvenser detta motsvarar (enligt ovan) och eftersom vi vet med vilken frekvens signalen är samplad kan vi överföra den normerade frekvensen till den riktiga frekvensen. Skriv

```
f=(0:M-1)/M*FT; plot(f,abs(ekgspek)), xlabel('frequency (Hz)'),
```

En observation är viktig att göra: Frekvenserna över halva samplingsfrekvensen (500 Hz) är bara en spegling av det som finns nedanför. Det är alltså nedan för halva samplingsfrekvensen som vi ska leta. Vi zoomar därför in den vänstra halvan (0-500 Hz och 0-800000 i amplitud).

```
axis([0 500 0 800000])
```

Det verkar som om den här signalen var ganska översamplad dvs samplingsteoremet var med råge uppfyllt. Inte mycket energi ligger över 50 Hz i spektrat. Vi zoomar därför ännu en gång fast nu både i amplitud och frekvens

```
axis([0 20 0 150000])
```

Det vi ser här är de frekvenser som den här EKG-signalen består av. Med består av menar vi att signalen matematiskt kan brytas ner i dessa frekvenser och inte att den har skapats genom att kombinera ett antal sinustoner med dessa frekvenser. Så är givetvis inte fallet; den här signalen har skapats genom att många celler samtidigt ändrade sin ytpotential och skapade ett elektriskt fält i bröstkorgen som vi mäter i en punkt med en elektrod. Som signalbehandlare har vi varit inkopplade på detta problem för att datorn automatiskt ska kunna leverera den exakta pulsen till läkaren och därför måste vi tolka vad vi ser i detta spektrum. Först ser vi att den frekvensen noll har högst energi. Detta orsakas av den glidande baslinjen som förvisso inte varierar med så kraftig amplitud jämfört med slagen men som är utspridd över hela signalen. Denna topp i spektrat är vi inte intresserade av. Sedan kommer nästa topp nästan exakt vid 1 Hz. Denna lägsta frekvens med övertoner på 2,3,4 Hz osv representerar den spetsiga vågformen hos hjärtslagen. Grundtonen för detta övertonsspektrum, dvs 1 Hz toppen, motsvarar repetitionsfrekvensen (den frekvens med vilken signalmönstret upprepar sig) hos signalen. Denna frekvens med vilken hjärtslagen upprepar sig mäter alltså pulsen. Vårt datorprogram ska alltså söka efter den första toppen från vänster i spektrat förutom den som eventuellt finns vid frekvensen noll för att hitta pulsen.

Uppgift 5: Slutligen ska vi undersöka vad ett enkelt filter kan åstadkomma på en EKG-signal. Vi börjar med att åter plotta vår signal men denna gången på övre halvan av en figur.

```
figure, subplot(211), plot(t,ekg1),
```

Vi bestämmer oss nu för att försöka dämpa ner bruset i denna signalen genom att säga att utsignalen vid tidpunkten n är ett skalat medelvärde av de femton senaste insignalsamplen. Filtret för detta är femton lika vikter, vilka t.ex. kan sättas till 0.2, för insignalerna $ekg1[n]$ till $ekg1[n - 14]$. Filtringen görs genom att falta (i MATLAB `conv`) insignalen med filtret

```
h=0.2*ones(1,15); y=conv(ekg1,h); y=y(15:length(y));
```

Faltningens längd blir ju signalens längd + filtrets längd - 1. Den sista raden tar bort insvängningsförloppet innan filtret har fått in data på alla sina tappar (vikter) och gör att utsignalen får samma längd som insignalen. Nu återstår att titta på resultatet

```
subplot(212), plot(t,y),
```

Det vi ser här är en signal med betydligt mindre brus. Nu är signalen så ren att vi till och med tydligt ser att det finns en liten topp innan var och en av de stora smala pulserna. Dessa små elektriska pulser motsvarar att förmaken drar ihop sig och fyller kamrarna med blod för att de ska vara fyllda när den stora sammandragningen av hjärtat kommer vid de stora pulserna.

7 Datorövning 3: Filtrering av bilder

Denna datorövningsyftar till att ge en förståelse av begreppet diskret frekvens. Som illustration används både bilder och endimensionella signaler. Den diskreta fouriertransformen introduceras, och begreppet frekvenssvar behandlas. Extra tonvikt är lagd vid kopplingen mellan frekvensinnehåll i en signal och signalens synliga egenskaper.

7.1 Datoruppgifter

Datoruppgift 1

Applicera ett glidande medelvärdesfilter på några olika vågformer. Testa med en fyrkantsvåg och några cosinussignaler och plotta resultaten. Nedan följer ett exempel på hur man kan skapa, filtera och plotta en fyrkantssignal.

```
>> N=256;
>> fhat=12/N; %Normerad frekvens för fyrkantssignalen
>> n=(1:N);xx=square(2*pi*fhat*n); %skapar en fyrkantssignal som insignal
>> L=5; %filterlängd
>> h=1/L*ones(L,1);
>> yy=conv(h,xx); %
>> subplot(211), stem(yy); %rita upp vektorn yy
>> subplot(212), stem(xx); %rita upp vektorn xx
```

Kan du hitta något samband mellan amplituden på svaret för olika frekvenser i insignalen. Relatera detta till resultaten i de teoretiska uppgifterna. Vad händer i början av utsignalen? I slutet?

7.2 Filtrering av bilder

En faltning kan illustreras med en mask som glider över signalen. För varje sampel där filtret och signalen överlappar multipliceras de motstående koefficienterna ihop och summan av dessa produkter bildar värdet i aktuellt sampel. För en mer detaljerad beskrivning av faltning, se boken sida 131. Det finns inget som hindrar att samma metodik tillämpas på tvådimensionella digitala signaler, exempelvis bilder. Vi skall nu undersöka vad den tvådimensionella motsvarigheten till filtret i föregående uppgift har för effekt på bilder. Ekvationen för den tvådimensionella faltningen är snarlik den endimensionella varianten. En varning är dock på sin plats: I bildsammanhang indexeras vanligen det mittersta elementet i masken som (0,0). Ett bildbehandlingsfilter behöver vanligen inte vara kausalt¹, en bild är ju normalt inte en tidssignal i den bemärkelsen!

Datoruppgift 2

Följande MATLAB program läser in en bild och beräknar det glidande medelvärdet för pixelvärdena.

```
>> xx=double(imread('lena.tif'))./255; %Läser in bilden, skalar pixelvärdena
>> N=9; %Sätter maskstorleken
>> hh=ones(N)./(N*N); % Skapar en kvadratisk mask med sidan N
>> yy=conv2(xx,hh); %Beräknar 2-D faltningen mellan bilden och filtret
```

Ni kan titta på en bild genom att använda `imagesc`. Om färgerna ser galna ut, skriv `colormap gray` och försök igen.

Prova att lägga brus på bilden genom att addera en matris med slumpstal. Filtera igen.

¹Ett filter är *kausalt* om utsignalen i varje tidpunkt endast beror på insignalen i samma tidpunkt (index) eller i tidigare tidpunkter

```
>> xx_brus=xx+rand(size(xx))*0.2; % skapa en brusig version av xx
```

- Hur påverkas en kant av filtret? Ett intervall med konstant signal?
- Vilka frekvenser dominerar en i en bildregion med konstant signal?
- Vilka frekvenser dominerar i en bild med mycket kanter och variationer?
- Hur påverkas en bild med störningar av filtret? Vilken typ av störningar kan man eliminera med denna typ av filter? Hur kan sådana störningar uppstå?

Gör gärna om filtreringen med varierande filterstorlek.

7.3 Ett FIR-filter för kantextraktion

Datoruppgift 3

Gör om datoruppgift 1 med nedanstående filter

$$h(n) = [00\bar{1}00] - \frac{1}{5}[11\bar{1}11] = [-\frac{1}{5} - \frac{1}{5} + \frac{4}{5} - \frac{1}{5} - \frac{1}{5}] \quad (1)$$

- Vad händer vid en kant?
- Vad händer med en signal som varierar mycket?
- Vad händer i en konstant signal region?
- Vilka diskreta ($\hat{f}$) insignal frekvenser förstärks respektive försvagas när det här filtret används?

Datoruppgift 4

Det är nu dags att utöka vår kantförstärkare till det tvådimensionella fallet. Även här kan vi skapa ett filter genom subtraktion av den utsmetade signalen.

$$h(k, l) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & \bar{1} & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & \bar{1} & 1 \\ 1 & 1 & 1 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} -1 & -1 & -1 \\ -1 & \bar{8} & -1 \\ -1 & -1 & -1 \end{bmatrix} \quad (2)$$

Applicera denna mask på samma bild som användes tidigare. **OBS!** Det finns ett par teknikaliteter som kan ställa till saker här. En av dem är att MATLAB:s `imagesc`-kommando skalar om pixelvärdena så att de passar in på färgkartan. Det kan vara värdefullt att också använda kommandot `image` för att få en känsla för hur effektivt vårt filter verkligen är.

- Vad händer med kanter?
- Vad händer med enfärgade ytor?
- Hur skulle filtret modifieras om enfärgade ytor inte helt skulle filtreras bort? I vilket sammanhang skulle det kunna vara användbart?

8 Datorövning 4: Filterdesign och digital audioteknik

Inledning

I denna laboration ska du bekanta dig med några grundläggande principer för digital audioteknik. Dessutom skall FIR-filter studeras och appliceras på några signaler. Sambandet mellan filtrets frekvenssvarfunktion och dess effekt på en signal skall undersökas. Målet är att ni efter laborationen skall:

1. Ha en känsla för sambandet mellan ett filters frekvensegenskaper och dess egenskaper i tid.
2. Förstå begreppet *kausalitet* för ett digitalt filter och vilka begränsningar det kan skapa vid implementation.
3. Förstå skillnaden mellan verklig och normaliserad frekvens för en samplad signal.
4. Kunna använda delar av MATLAB:s *signal processing toolbox* för filtrering av signaler i en och två dimensioner.
5. Ha fått en första titt på användningen av en digital signalprocessor (DSP).

Laborationen består av tre delar: förberedelseuppgifter (som skall vara gjorda före laborationstillfället), laborationsmoment med signalbehandling i Matlab samt realtidssignalbehandling med DSP och programpaketet Code Composer Studio (CCS) från Texas Instruments tillsammans med hårdvaran DSK 6713 DSP kit.

Till de olika laborationsmomenten finns illustrationer som motsvarar det system som körs på DSK 6713.

Allt arbete under laborationens gång sker med utgångspunkt från Matlab. Då DSK6713 ska användas, sker anropet från Matlab. Ni kan arbeta parallellt i Matlab även då program körs i DSPn. Program laddas ner till DSPn genom att trycka på knappen (**Open DSP**). Avsluta programmet i DSPn (**Close DSP**) innan nerladdning av nytt program.

Förberedelseuppgifter

Förberedelseuppgift 1

I laborationen kommer vi att titta på filter som ger fördröjning och eko. Beräkna frekvenssvaret för filterna som definieras av följderna nedan:

$$h(n) = \frac{1}{3}[1 \ 0 \ 0 \ 0 \ 1] \quad (3)$$

$$(4)$$

För att göra detta kan följande formel användas:

$$\mathcal{H}(\hat{\omega}) = \sum_k h(k)e^{-j\hat{\omega}k} \quad (5)$$

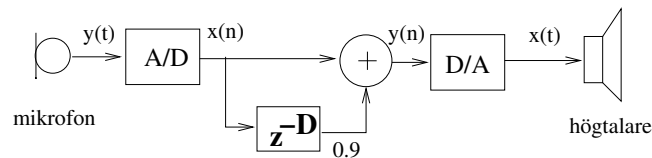
Förberedelseuppgift 2

Skissa frekvenssvaret för filtret ovan. Använd gärna MATLAB. Vilket $\hat{\omega}$ -intervall är lämpligt att använda?

Matlabuppgifter

FIR-filter i tidsplanet

Vi börjar med att titta på ett exempel med eko i matlab. För att åstadkomma ekoeffekten använder vi oss av nedanstående koppling (z^{-D} fördröjer signalen D sampel).



Simulera detta i MATLAB och lyssna på effekten.

Exempel på MATLABkod:

```
t=-1:0.0001;1; %generera 2 s lång tidsvektor, FS=1/10000
x=vco(t,[300,800],10000); %generera en svept sinussignal mellan 300 och 800 Hz
soundsc(x,10000); %lyssna
noll=zeros(1,500); %generera 500 nollor
x1=[x,noll];x2=[noll,0.9*x];
y=x1+x2; %generera ekot
soundsc(y,10000); %lyssna, högtalare eller hörlurar måste vara anslutna
```

Välj nytt x (samplat tal och lyssna på ekoeffekten).

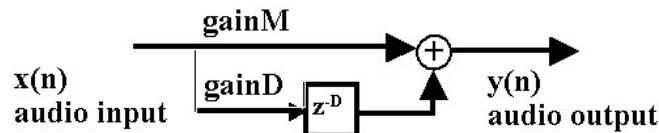
8.1 Digital Audioteknik i realtid

Uppgift 1

Ni skall i denna del av laborationen bekanta dig med det enklaste byggblocket som används för att generera analoga ljudeffekter med hjälp av digital signalbehandling, ekoprocessorn.

Skriv först `init.digsig` i Matlab.

Ekoprocessorn fördröjer signalen D sampel och multiplicerar den med en dämpningsfaktor a , $|a| < 1$. Genom att addera den resulterande signalen till den ursprungliga, fås en ekoeffekt. Du ska nu undersöka det system som visas i nedanstående figur. För att starta programmet och processen på DSPn, skriver du `echo_menu`.



Du ändrar fördröjningen med reglaget **Delay**, och dämpningen med **Gain**. Prata i mikrofonen och lyssna!

Vid vilken fördröjning blir det störande att prata. Testa genom att dra ner **gainM** till noll och bestäm den delay vid vilken du besväras av ekot vid tal (Prata i mikrofonen och lyssna i hörlurarna).

Svar: Jag tycker det blir störande vid ms delay.

Teckna nu ett uttryck för hur utsignalen $y(n)$ beror på tidigare ut- och insignaler $x(n)$, $y(n)$. Hur är nollställena placerade? Hur ser systemets överföringsfunktion (systemfunktion), $H(z)$, ut?

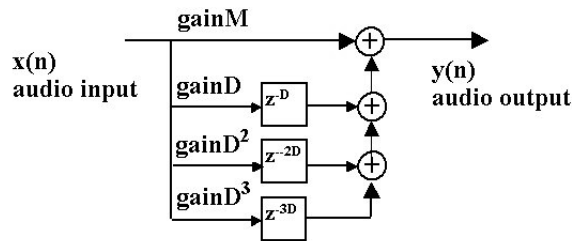
Vilken relation finns mellan antal nollställen och antal sampel fördröjning?

Har systemet linjär fas? Motivera!

Avbryt Matlab-programmet med `exit`.

Uppgift 2

Som du märkte i föregående uppgift så blir det ett lätt syntetiskt eko, utan någon "klang". Genom att koppla samman flera ekoprocessorer kan man generera något som benämns efterklang. Fördröjningarna och dämpningarna väljs som multipler av basvärdet D respektive som potenser av basvärdet a .



Anropa **echo_menu** och välj tre delay lines för att starta ett system enligt figur ovan. Justera basvärdena för fördröjningen (D) och dämpningsfaktorn (a) med reglagen **Delay** respektive **Gain**. Prata i mikrofonen och lyssna på resultatet.

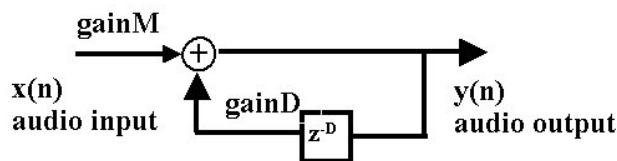
Hur ser systemets impulssvar ut? Bestäm $h(n)$ med ledning av figuren ovan. Ledning: impulssvar är just svaret på en impuls!.

Om man låter antalet ekoprocessorlänkar bli väldigt stort, hur kan man då modellera efterklang? *Skriv ut N termer av överföringsfunktionen, och summera. Observera att $|a| < 1$.*

Stäng programmet (programmet i DSP körs tills nästa program laddas ned).

Frivillig uppgift

Realisera ett system motsvarande oändligt många ekoprocessorlänkar. Ledning: vad händer om man kör en eko-signal tillbaka genom ekoprocessorn?



Testa genom att köra **echo_menu** och välja IIR-filer.

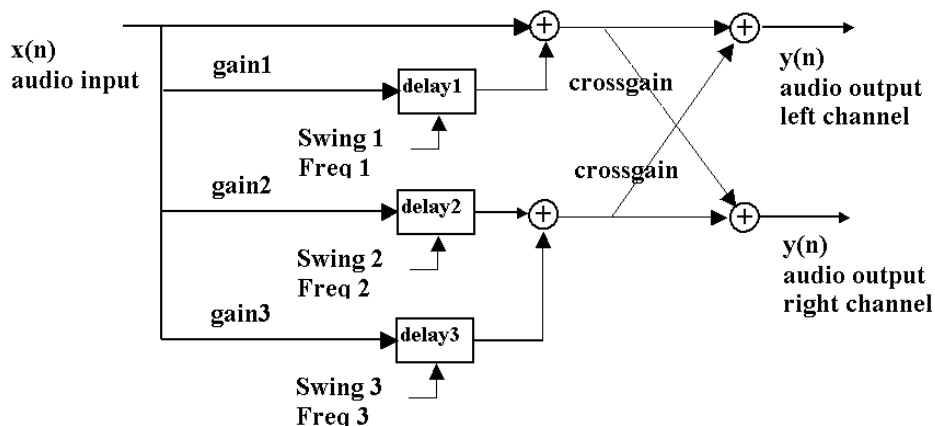
Uppgift 3

Då en vältränad kör sjunger uppfattar man det som att alla sjunger i takt. Dock så ligger varje körmedlem något ur fas både vad röstläge och takt beträffar. Det är just detta som ger en fyllig köreffekt.

För att skapa köreffekt från en monokanal kan man utgå ifrån ekoprocessorer, vars tidsfördröjningar man låter variera i tiden. Dessutom kan man skapa en illusion av stereoljud, genom att dela in ekoprocessorerna i två grupper, som går till vardera högtalare. Då man spelar in ljud från , t.ex., en kör, med två mikrofoner, kommer vardera mikrofon att registrera lite av det ljud som huvudsakligen registreras av den andra mikrofonen. För att återskapa detta kan man låta summasignalen från vardera grupp av ekoprocessorer dämpas och adderas till den andra gruppens signal.

Figuren nedan beskriver en köreffektsgenerator bestående av två grupper. De tidsvariabla fördröjningarna består av en konstant del, **Offset**, och en tidsvariabel del, som styrs av

sinusar med amplitud **Swing** och frekvens **Freq**. Varje gren i ekoprocessorn har en variabel förstärkning, **Gain**. Summasignalen från vardera grupp viktas med **CrossGain**, och adderas till den andra gruppen. Resultatet hörs i högtalarna, samt visas grafiskt i ett fönster.



Anropa `echo_chorus_menu` för att starta ovanstående modell i DSP:n. Börja t.ex. med att justera kanalerna var för sig, genom att dra ner **Gain** för övriga kanaler. Ändra fördröjningsmodulationen genom att justera sinusens amplitud och frekvens, **Swing** respektive **Freq**, samt den konstanta delen, **Offset**. **Observera att Swing måste vara mindre än Offset**. Då alla kanaler är justerade, drar du upp förstärkningarna till ca 50%. Justera därefter även **CrossGain**.

Det kan vara svårt att hitta riktigt bra inställningar, och eftersom parametervärderna bl.a. beror på din röst, är det svårt att ge några exakta riktvärden. Dock kan det nämnas att för att få ett fylligt ljud bör kanalerna ha olika konstanta fördröjningar (**Offset**), vilket ger en efterklangseffekt, samt att fördröjningsmodulationen inte ska vara för kraftig (**Swing**, **Freq**). Prova olika inställningar! *Överkurs: Sjung!*

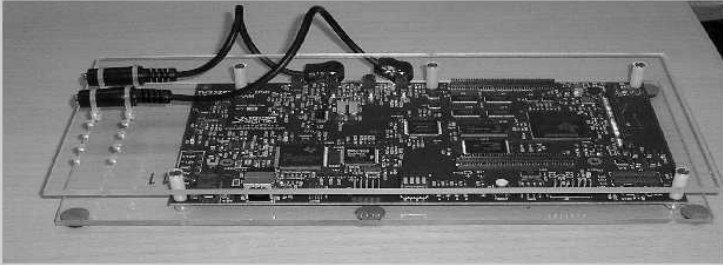
Du är nu klar med laborationen. Kontakta laborationshandledaren för att diskutera uppgifterna.

Program i DSP:n

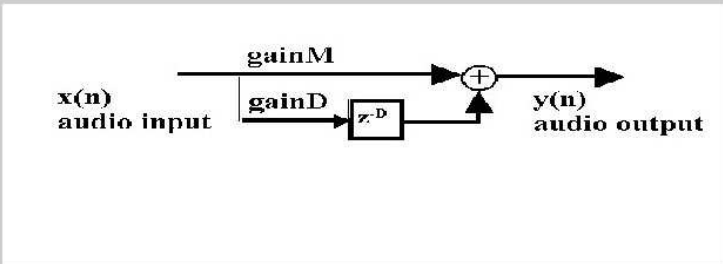
Beskrivning av echo_menu

Programmets funktioner beskrivs i korthet nedan.

Laboratory work, Echo processor,



Block diagram,



Delay parameters

Delay (ms): 20

GainM: 1

GainD: 0

No of delay units

1

FIR

Download

Open DSP

Close DSP

Exit

Kommando:

`global cc, echo_menu`, cc är Matlab-address till DSPn (global).

Funktioner

Download Läser ner parametrar till DSPn.

Open DSP Startar CCS, öppnar Matlab-länk till DSPn, laddar ner och startar program i DSPn.

Close DSP Stänger DSP (Stänger Matlab-länken till DSPn. Programmet i DSP fortsätter tills nytt program laddas ner).

FIR Väljer FIR-filter (1 till 3 delay lines).

IIR Väljer IIR-filter (1 delay line).

Exit Avslutar Matlab-programmet (Matlab-länken och programmet i DSPn påveckas ej).

Beskrivning av echo_chorus_menu

Programmets funktioner beskrivs i korthet nedan.



Kommando:

`global cc, echo_chorus_menu`, `cc` är Matlab-adress till DSPn.

Funktioner

Download Läser ner parametrar till DSPn.

Open DSP Startar CCS, öppnar Matlab-länk till DSPn, laddar ner och startar program i DSPn.

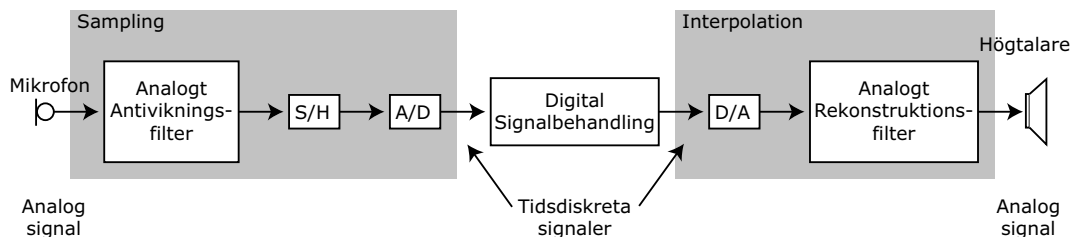
Close DSP Stänger DSP (Stänger Matlab-länken till DSPn. Programmet i DSP fortsätter tills nytt program laddas ner).

Exit Avslutar Matlab-programmet (Matlab-länken och programmet i DSPn påverkas ej).

9 Datorövning 5: Tal och röstanalys

9.1 Ett system för digital behandling av analog signal: Exempel Signalbehandling av talsignaler

I det här fallet är det en analog signal som spelas in, processas digitalt och spelas upp analogt.



Figur 4: System för digital behandling av analog signal: Exempel Signalbehandling av talsignaler.

Detta system är klassiskt och det var så här digital signalbehandling slog igenom från början. Som nämntes i inledningen till datorövningarna finns det många fördelar med att processa signaler digitalt. Vi repeterar återigen funktionen för de olika blocken: En analog sensor t.ex. en mikrofon spelar in en analog signal. En viss samplingsfrekvens bestäms och ett antivikningsfilter ser till att inga frekvenser över halva samplingsfrekvensen släpps in i systemet innan övergången till en digital signal. Den digitala signalen kan nu processas digitalt på olika sätt innan den förs tillbaka till den analoga världen i interpolatorn. Den analoga signalen kan nu åter spelas upp eller sändas vidare.

I den här datorövningen ska du öka din förmåga att tolka signaler och system i termer av frekvenser, poler och nollställen. Teorin illustreras här med hjälp av talsignaler och din uppgift är att i MATLAB analysera dessa, tolka dem och påverka dem innan du skickar vidare signalen för uppspelning. Signalerna är inspelade/samplade tidigare, den digitala signalbehandlingen utförs i MATLAB och interpolationen sker i datorns ljudkort.

9.2 Några talsignaler

Digital signalbehandling av talsignaler har under senare år varit mycket populärt. Allt fler applikationer finns på marknaden vad gäller allt från talsyntes och talanalys till talkompression/modellering och röststyrning. Alla dessa signalbehandlingstillämpningar har en sak gemensamt och det är att de bygger på matematiska modeller av hur rösten fungerar. Sammanfattningsvis kan man säga att stämbanden för varje vokal producerar en likriktad sinus med en period motsvarande personens grundton för tal, den s.k. *pitchfrekvensen*. Hade detta varit en ren sinus hade den bara innehållit en frekvens men nu är det en avklippt sinus och då tillkommer en massa övertoner (*formanter*) till grundtonen. Detta ljud fortplantar sig sedan genom hals och munhåla och passerar ut genom munnen. Beroende på hur man formar munhålan och munnen förstärks vissa frekvenser (munhålans resonansfrekvenser). Detta bestämmer i princip vilken vokal som kommer ut ur munnen. Munhålan och munnen kan modelleras som ett filter på ljudet från stämbanden eftersom de just förstärker vissa frekvenser. Detta filter har då poler i munhålans resonansfrekvenser.

9.3 MATLAB-uppgifter

Uppgift 1: Ladda ovan angivna filer genom att skriva

```
load man1;
load kvinna1;
```

Uppgift 2: Vi ska börja med att lära oss lite mer om talsignaler. Detta gör vi genom att först lyssna och sedan plotta våra två signaler.

```
FT=8000;
soundsc(man1,FT),
soundsc(kvinna1,FT),
figure,
subplot(211),plot(man1),
subplot(212), plot(kvinna1),
```

I den här skalan ser vi ingen strukturell skillnad mellan män och kvinnors tal. Det vi ser är att talsignalen ibland har mycket hög amplitud och ibland ingen alls. Det är vokaler som har hög amplitud och konsonanterna är svåra att skilja från brus eller tystnad. Av denna anledning fokuseras nästan all talsignalbehandling t.ex. talkompression och röst styrning på de ljud som innehåller toner dvs vokaler. Om man tittar i den nedre plotten är det nu lätt att identifiera de olika stavelserna. Från vänster i signalen blir detta: four-teen, fif-teen, six-teen, se-ven-teen osv.

Uppgift 3: För att lära oss mer om talet måste vi alltså lära oss mer om vokalljud. Vi klipper därför ut några ljud

```
e_man1=man1(10401:11200); i_man1=man1(8201:9000);
e_kvi1=kvinna1(6201:7000); o_kvi1=kvinna1(3801:4600);
```

och lyssnar på dessa med hjälp av

```
soundsc(e_man1,FT);
soundsc(i_man1,FT);
soundsc(e_kvi1,FT);
soundsc(o_kvi1,FT);
```

Vi hör tydligt att kvinnans e-ljud ljuder med en mycket högre frekvens än mannens. Spektrum för dessa signaler skulle klarlägga dels skillnaden mellan män och kvinnors tal och skillnaden mellan olika vokaler. Vi delar upp spektrat i lika många punkter som signalen var lång.

```
figure;
N=800;
subplot(221); plot((0:N-1)/N*FT,abs(fft(e_man1,N)));axis([0 1500 0 300]);
subplot(222); plot((0:N-1)/N*FT,abs(fft(i_man1,N)));axis([0 1500 0 300]);
subplot(223); plot((0:N-1)/N*FT,abs(fft(e_kvi1,N)));axis([0 1500 0 300]);
subplot(224); plot((0:N-1)/N*FT,abs(fft(o_kvi1,N)));axis([0 1500 0 300]);
```

Vi ser nu att mannens röst för båda ljuden har en grundton (pitchfrekvens) på ca 150 Hz medan kvinnans röst ligger nästan dubbelt så högt i frekvens. Vi kan också dra slutsatsen att när munnen formas för ett e (de båda vänstra plottarna) så skapas resonans (förstärkning) av frekvenser kring 300 Hz med mycket lite över toner däröver. När munnen istället formas för ett i (övre högra plotten) förstärks frekvenser strax under 500 Hz med ganska lite förstärkning däröver och när munnen formas för o (nedre högra plotten) förstärks också frekvenser runt 500 Hz men nu också frekvenser runt 1000 Hz. Så här kan man maskinellt tolka tal: Om talet först har förstärkning kring 500 Hz men ej över (i) och ett antal milisekunder senare förstärkning runt 300 Hz och inget däröver (e) så kanske ordet ska vara bil-en". Med detta system hade inte varit lätt att separera detta från pil-en" vilket ofta är svårt för sådana här system.

Uppgift 4: Nu ska vi ändra i de inspelade ljuden och skicka dem vidare för uppspelning. Vi väljer två enkla FIR filter: Ett lågpasfilter och ett högpasfilter.

```
lowpass=[1 1 1 1]; highpass=[1 -1 1 -1];
denominator=1 %Ej IIR
man1_l=filter(lowpass,denominator,man1);
man1_h=filter(highpass,denominator,man1);
kvinna1_l=filter(lowpass,denominator,kvinna1);
kvinna1_h=filter(highpass,denominator,kvinna1);
```

Vi spelar upp dessa i följd efter den ursprungliga

```
soundsc([man1;man1_l;man1_h],FT),
soundsc([kvinna1;kvinna1_l;kvinna1_h],FT),
```

Uppgift 5: Slutligen provar vi att lägga på ett enkelt så kallat reverb (efterklang) på kvinnans röst för att få den att låta som om hon pratar i t.ex. en kyrka. Vi använder då ett filter som förutom den ursprungliga signalen ska eka några gånger inom ca 50 ms.

```
ekofilter=[1 zeros(1,250) 0.8 zeros(1,250) 0.6];
```

Detta filter ekar en gång med faktorn 0.8 efter 250 sampel (ca 30 ms) och en gång med faktorn 0.60 efter ytterligare 250 sampel. Skriv in och lyssna.

```
klang_kvinna1=filter(ekofilter,1,kvinna1);
soundsc([kvinna1;klang_kvinna1],FT)
```


10 Datorövning 6: IIR-filter

10.1 Inledning

I denna laboration ska du bekanta dig med de grundläggande principerna för digital audioteknik. Dessutom ska du studera hur poler och nollställen tillsammans bygger upp IIR-filter. Målet är att du skall:

1. Få en känsla för hur polernas och nollställets placering påverkar egenskaperna för IIR-filter.
2. På ett lekfullt sätt bekanta dig med enkel digital audioteknik i realtid på DSP.

Laborationen består av tre delar: förberedelseuppgifter (som skall vara gjorda före laborationstillfället), laborationsmoment med signalbehandling i Matlab samt realtidssignalbehandling med DSK6713.

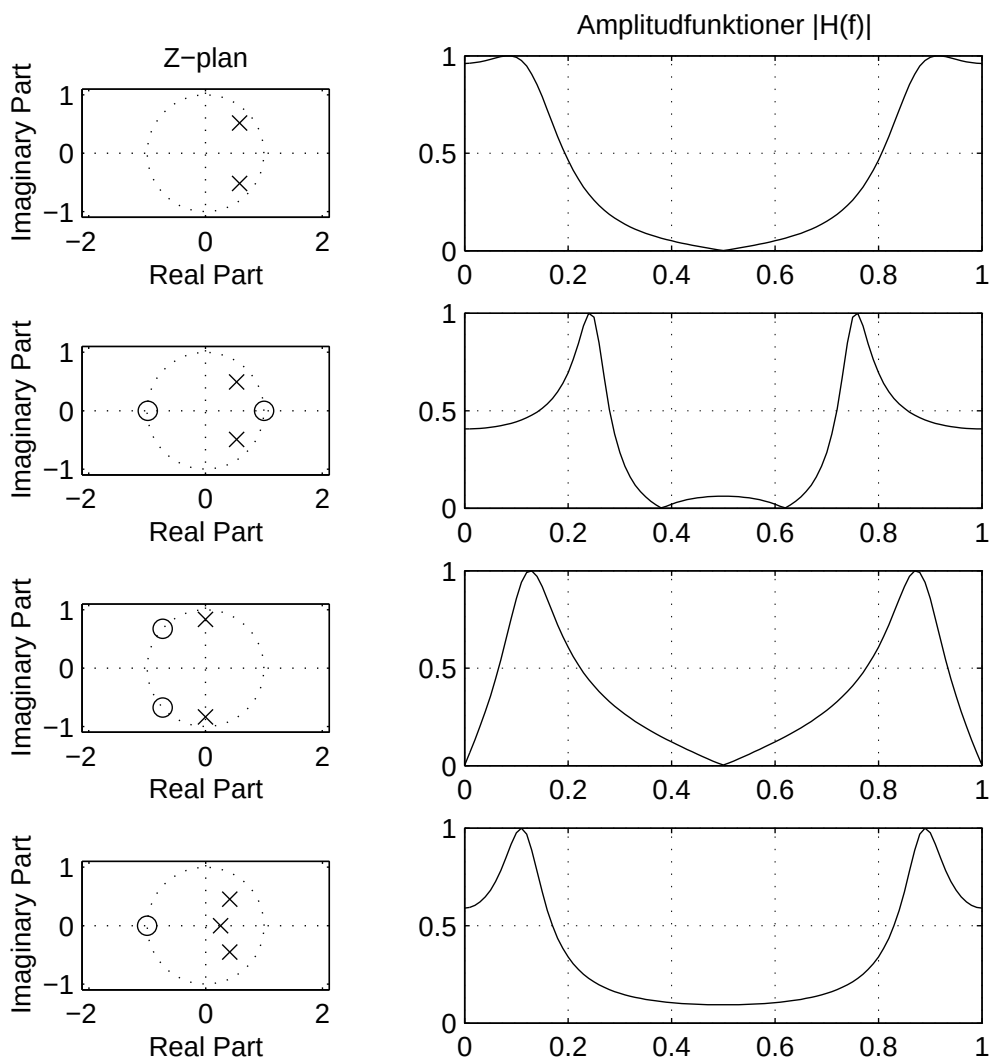
Allt arbete under laborationens gång sker med utgångspunkt från Matlab. Då DSK6713 ska användas, sker anropet från Matlab. Ni kan arbeta parallellt i Matlab även då program körs i DSPn. Program laddas ner till DSPn genom att trycka på knappen (**Open DSP**). Avsluta programmet i DSPn (**Close DSP**) innan nerladdning av nytt program. Parametrar till programmet i DSP laddas ner med **Download**

10.2 Förberedelseuppgifter

Vi beskriver ofta ett tidsdiskret system $H(z)$ med dess poler och nollställen som vi illustrerar i ett z -plan. Utgående från polerna och nollställena vill vi uppskatta frekvensfunktionen $H(f)$. Frekvensfunktionen $H(f)$ är värdet av $H(z)$ på enhetscirkeln och som en tumregel kan vi utnyttja att $|H(f)|$ är litet nära ett nollställe och stort nära en pol.

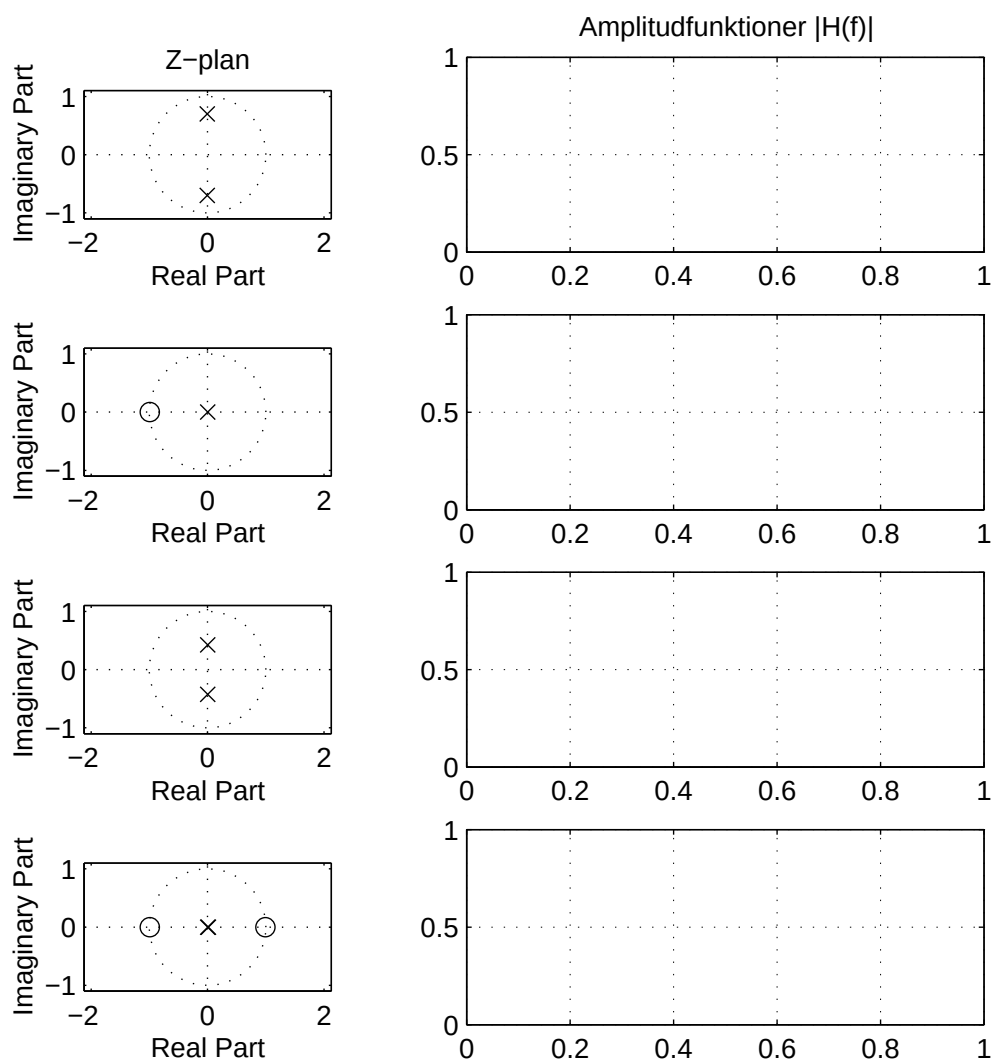
Förberedelseuppgift 1

Utnyttja ovanstående resonemang för att para ihop nedanstående fyra kombinationer av pol-nollställesdiagram med rätt amplitudfunktion $|H(f)|$.



Förberedelseuppgift 2

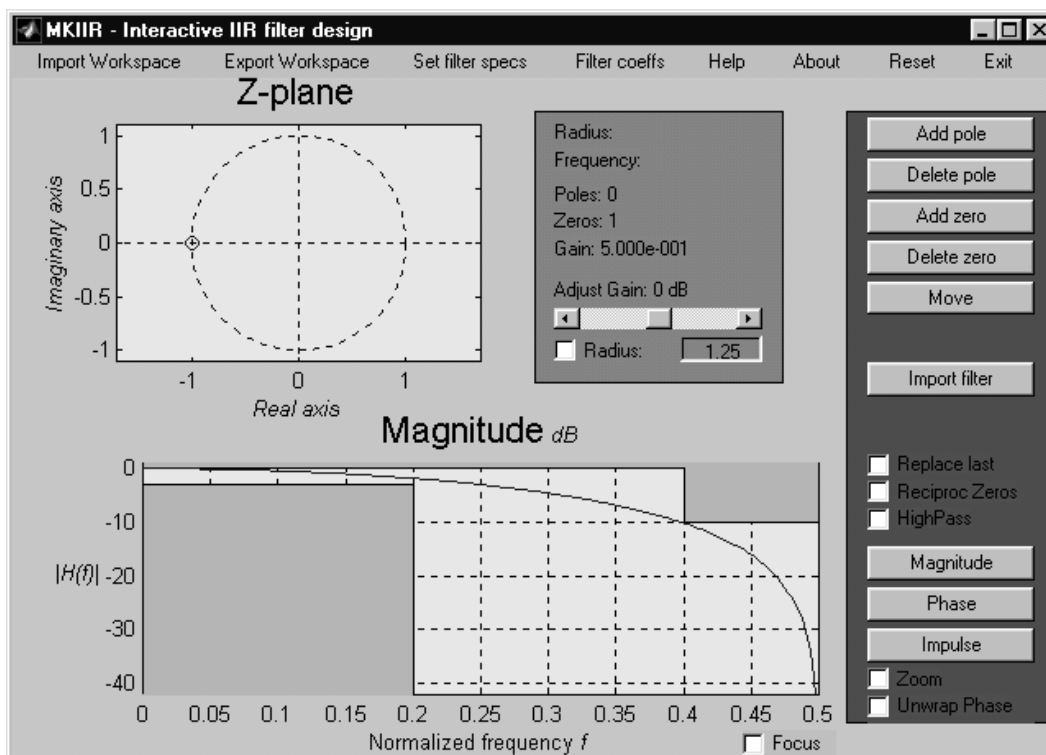
Utnyttja erfarenheten från förberedelseuppgift 1 och skissa amplitudfunktionen $|H(f)|$ för nedanstående fyra exempel på pol-nollställesdiagram.



10.3 Laborationsuppgifter

IIR-filter

Programmet MKIIR möjliggör interaktiv design av IIR-filter, genom utplacering av poler och nollställen i z-planet med hjälp av muspekaren. Amplitudfunktion, fasfunktion och trunkerat impulssvar kan undersökas. Om kravspecifikation är angiven kan standardfilter från Matlab Signal Processing Toolbox importeras. Ett exempel på skärmbild från MKIIR visas nedan. Programmets funktioner beskrivs i korthet i slutet av handledningen.



Uppgift 1

I denna uppgift ska du undersöka hur impulssvaret påverkas då du placerar en pol eller ett komplexkonjugerat polpar i z-planet. Skriv MKIIR för att starta IIR-filterdesignprogrammet.

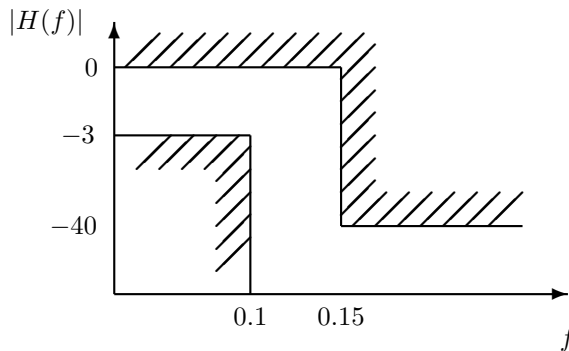
Börja med att markera funktionen **Replace last**, som gör att senast inmatad pol ersätts då en ny pol läggs ut. Undersök amplitudfunktionen samt det trunkerade impulssvaret då du flyttar polen längs den positiva delen av reella axeln. Kan du se något beroende mellan polens avstånd till enhetscirkeln och hur snabbt impulssvaret klingar av?

Flytta runt polen i enhetscirkeln. Ser du något samband mellan frekvensen (motsvarar polens vinkel till origo) och impulssvarets utseende? Undersök även amplitudfunktionens utseende.

Då du har noterat förhållandet mellan polens placering och utseendet hos impulssvaret, avmarkera **Replace last**.

Uppgift 2

Du ska nu genom placering av nollställen och poler försöka uppfylla nedanstående kravspecifikation. Tryck **Import Workspace** och öppna filen `iirspec1.mat` för att läsa in specifikationen i MKIIR.



Försök nu att uppfylla kravspecifikationen med så få poler och nollställen som möjligt. Tänk på att filtrets amplitudfunktion beror på avståndet mellan enhetscirkeln och nollställena, respektive avståndet mellan enhetscirkeln och polerna. Den punkt på enhetscirkeln varifrån avståndet mäts beror på den normerade frekvensen f . Som tumregel kan man utnyttja amplitudfunktionen är liten nära ett nollställe och stort nära en pol. Använd samma antal nollställen som poler.

Då du har lyckats uppfylla specifikationen ska du jämföra med hur nollställen och poler placeras av några vanliga filtertyper. Spara ditt filter genom att trycka **Export Workspace** och därefter ange filnamn, t.ex. `filter1.mat`. Välj därefter **Import filter**.

Om du vill så kan du ändra kravspecifikationen genom att välja **Set filter specs** och där ange nya gränser. Undersök t.ex. hur filtrens ordning påverkas då övergångszonen (mellan f_p och f_s) blir smalare.

Frivillig uppgift

I litteraturen finns ett stort antal filtertyper beskrivna. Några av dessa finns också i MKIIR. Undersök de filtertyper som finns i menyn och studera hur de olika filtertyperna placerar ut polerna och nollställena för att uppfylla kravet på dämpning. Om du vill jämföra med ditt eget filter, läs in detta genom att ta **Import Workspace** och välja den fil som du lagrade tidigare.

Avsluta MKIIR genom att trycka på **Exit**.

Uppgift 3

De filter som du har undersökt ovan karakteriseras av passband och stoppband. I vissa sammanhang vill man endast spärra en viss frekvens, och låta resten passera opåverkat. Ett sätt att ta bort en specifik frekvens är att designa ett filter som lägger ut nollställen på enhetscirkeln, vid de vinklar som motsvarar den oönskade frekvensen. *Eftersom en reell sinus med frekvens f_0 har en komponent vid frekvensen $f = -f_0$ och en vid $f = f_0$, måste man lägga ut ett komplexkonjugerat par av nollställen.*

Nackdelen med att endast använda nollställen på enhetscirkeln, vilket motsvarar ett FIR-filter där polerna ligger i origo, är att ett stort område kring den oönskade frekvensen

påverkas. För att uppnå en i det närmaste konstant förstärkning förutom vid den frekvens som man vill ta bort, flyttar man ut polerna från origo och placerar dem **strax innanför** nollställena. Dessa typer av filter kallas *notch filter*, och har systemfunktionerna

$$H_{FIR}(z) = (1 - e^{j\omega_0} z^{-1})(1 - e^{-j\omega_0} z^{-1}) = 1 - 2 \cos(\omega_0) z^{-1} + z^{-2}$$

$$H_{IIR}(z) = \frac{(1 - e^{j\omega_0} z^{-1})(1 - e^{-j\omega_0} z^{-1})}{(1 - r e^{j\omega_0} z^{-1})(1 - r e^{-j\omega_0} z^{-1})} = \frac{1 - 2 \cos(\omega_0) z^{-1} + z^{-2}}{1 - 2r \cos(\omega_0) z^{-1} + r^2 z^{-2}}$$

där ω_0 är den vinkelfrekvens som ska blockeras, och r är polernas radie.

Du ska nu kontrollera hur vinkelfrekvensen polernas radie påverkar filtrens amplitudfunktion. Undersök nu filtrets amplitud- och fasfunktion genom att använda makrot **freqz** i Matlab Signal Processing Toolbox. För att kunna använda **freqz** måste du omvandla filtret från pol/nollställe-representation till polynomform med hjälp av funktionen **poly**. Är du osäker på hur **freqz** och **poly** fungerar, använd **help** i Matlab. *Observera att i Matlab Signal Processing Toolbox normeras frekvenserna annorlunda än i kursen.*

Börja med att ändra frekvensen, men låt polerna ligga kvar i origo. Studera amplitudfunktionen. Öka nu polernas radie, och undersök hur amplitudfunktionen påverkas.

Uppgift 4

I denna uppgift ska du ta bort störningar från en ljudfil. Störningarna är sinussignaler som är utlagda vid tre frekvenser. Ljudfilen består av två kanaler (stereo) och är samplad med **8000 Hz**. Störningarna är identiska på de båda kanalerna.

Anropa **jukebox** (Matlab; global S, jukebox) och välj en låt från listan. Variablen S kommer nu att innehålla 100 000 sampel från din valda låt. Vill du lyssna på låten så gör det med hjälp av Windows Mediaplayer (eller liknande). Unvik att köra ut ljud i Matlab då det inte går att avbryta uppspelningen.)

Nu ska du ta fram störningarnas frekvenser. Anropa **MKSA**. Välj funktionen **PEAK** och markera störningarnas toppar med muspekaren. *Störningarna är utlagda på multipler av 10 Hz. Om du t.ex. läser av 1238 Hz, så ska du använda 1240 Hz i dina beräkningar.* Avsluta **MKSPA** med **Exit** när du har noterat frekvenserna.

Din uppgift är nu att konstruera ett notchfilter som tar bort de oönskade tonerna. Detta gör du genom att ange filtrets nollställen i vektorn **Z**, och dess poler i vektorn **P**. *Tänk på att $e^{-j\omega}$ beräknas med funktionen $\exp(-j*\omega)$ i Matlab.* Då du är klar ska du ha följande vektorer i arbetsytan,

$$\begin{aligned} Z &= [z_0 \ z_1 \ \dots \ z_5]; \\ P &= [p_0 \ p_1 \ \dots \ p_5]; \end{aligned}$$

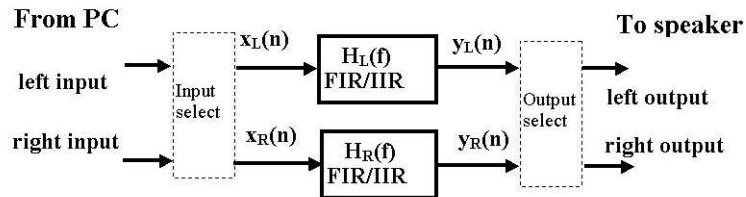
$$\begin{aligned} B &= \text{poly}(Z); \\ A &= \text{poly}(P); \end{aligned}$$

Observera att du måste använda j som imaginär enhet, eftersom i används som räkneindex i flera makron.

Undersök nu filtrets amplitud- och fasfunktion genom att använda makrot **freqz** i Matlab Signal Processing Toolbox på samma sätt som tidigare. För att kunna använda **freqz** måste du omvandla filtret från pol/nollställe-representation till polynomform med hjälp av funktionen **poly**. Är du osäker på hur **freqz** och **poly** fungerar, använd **help** i Matlab.

Observera att i Matlab Signal Processing Toolbox normeras frekvenserna annorlunda än i kursen.

När filtret uppfyller dina önskemål ska du ladda ner det till DSPn. Skriv `filter_menu` (global cc B A, `filter_menu`). Tryck sedan `Open DSP` och vänta på att DSPn startas och programmet laddas ner. Starta din valda låt i t.ex. Windows Media Player och testa att filtret har önskad funktion genom att ladda ner ditt filter med `Download`. Prova både FIR och IIR-filtret.



Sammanfattning

I denna del har du undersökt hur poler och nollställen samverkar. Tidsdiskreta IIR-filer bygger ofta på analoga filter (Chebyshev, Butterworth, Elliptiskt). Trots att det kan verka gammalmodigt att utgå från analoga filter så är detta vanligt. Det kan vara bra att känna till att de tidsödande beräkningarna kan utföras av Matlab Signal Processing Toolbox, med ett par enkla kommandon för respektive filtertyp. Genom att skriva `help signal` får du fram de funktioner som ingår i Matlab Signal Processing Toolbox. Alternativt kan du välja `Import filter` och `Info` i MKIIR, för att få en lista på de aktuella kommandona.

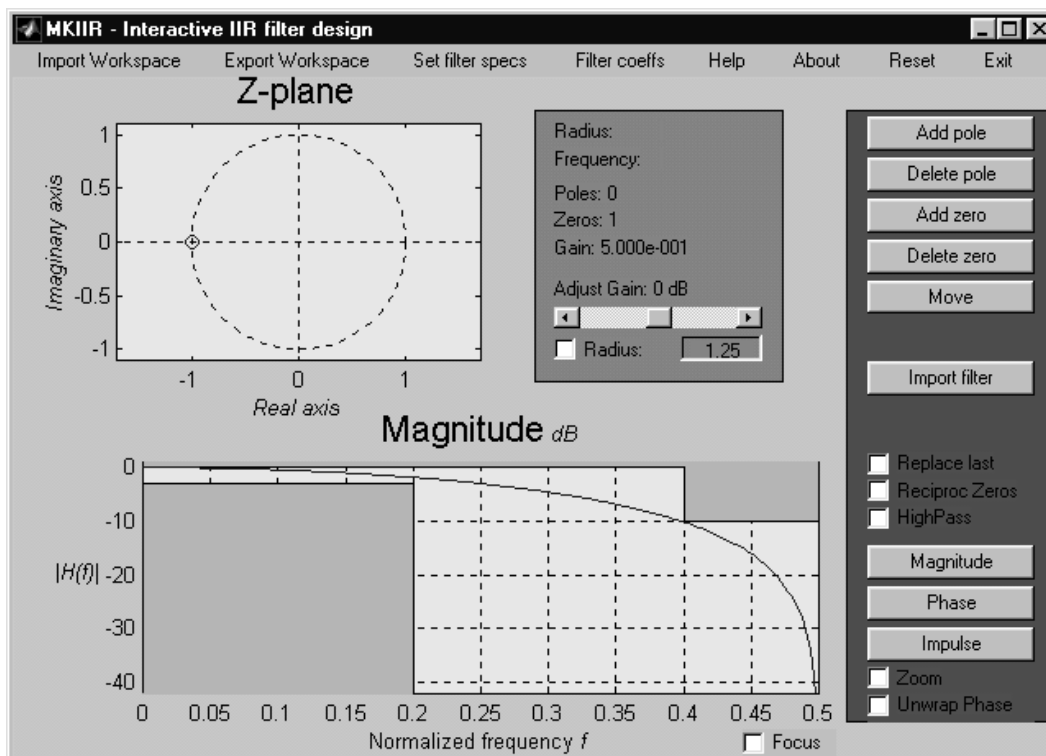
Notchfilter kan bl.a. användas för att ta bort störningar från elnätet (50 Hz), och för att ta bort periodiska störningar från t.ex. motorer, när varvtal är känt.

10.4 Programbeskrivningar

IIR-filterdesign, MKIIR

Beskrivning

MKIIR möjliggör interaktiv design av IIR-filter, genom utplacering av poler och nollställen i z-planet med hjälp av muspekaren. Amplitudfunktion, fasfunktion och trunkerat impulssvar kan undersökas. Om kravspecifikation är angiven kan standardfilter från Matlab Signal Processing Toolbox importeras. Programmets funktioner beskrivs i korthet nedan.



Funktioner

Import Workspace Inläsning från fil av kravspecifikation, och i förekommande fall, filter.

Export Workspace Lagring till fil av kravspecifikation, och i förekommande fall, filter.

Set filter specs Inmatning av kravspecifikation; Passbands- och spärbandsfrekvens (normerad frekvens), passbandsrippel och spärbandsdämpning (dB). För att helt ta bort kravspecifikation, ange samma passbands- och spärbandsfrekvens.

Filter coeffs Inmatning eller utskrift av filterkoefficienter.

Help Ej implementerad ännu.

About Information om programversion.

Reset Rensar alla poler och nollställen, men bevarar eventuell kravspecifikation.

Exit Avslutar programmet. Sparar arbetsytan till filen `current.mat`.

Add pole Utplacering av poler i z-planet. Peka på området innanför enhetscirkeln, tryck vänster musknapp för att placera pol. För att avsluta funktionen, tryck höger musknapp.

Delete pole Borttagning av poler i z-planet. Peka på den pol som ska tas bort, tryck vänster musknapp. För att avsluta funktionen, tryck höger musknapp.

Add zero Utplacering av nollställena i z-planet. Peka på området på eller innanför enhetscirkeln, tryck vänster musknapp. För att avsluta funktionen, tryck höger musknapp.

Delete zero Borttagning av nollställena i z-planet. Peka på det nollställe som ska tas bort, tryck vänster musknapp. För att avsluta funktionen, tryck höger musknapp.

Move Flytta poler eller nollställena i z-planet. Peka på pol eller nollställe som ska flyttas, tryck vänster musknapp. Det valda objektet markeras med svart. Peka på ny position, tryck vänster musknapp. För att avsluta funktionen, eller avbryta omplacering, tryck höger musknapp.

Import filter Import av standardfilter från Matlab Signal Processing Toolbox. Förutsätter att kravspecifikation finns. Observera att Butterworthfiltret i Matlab Signal Processing Toolbox anpassas efter spärbandsdämpningen, och ej nödvändigtvis skär genom -3 dB-punkten.

Magnitude Visar filtrets amplitudfunktion.

Phase Visar filtrets fasfunktion.

Impulse Visar filtrets impulssvar. För IIR-filter visas det trunkerade impulssvaret.

Replace last Det senast utlagda nollstället tas bort då ett nytt läggs ut. Fungerar på motsvarande sätt för poler.

Reciprocal Zeros Anger att reciproka nollställena ska läggas ut. Fungerar endast för nollställena.

HighPass Anger att filtret ska vara av högpasstyp. Styr den konstanta förstärkningen. För högpasfilter gäller att $|H(f)|_{|f=0.5} = 1$, och för lågpasfilter att $|H(f)|_{|f=0} = 1$.

Zoom Anger att Matlabs zoom ska vara aktiverad.

Unwrap Phase Anger att faser ska ritas upp utan att begränsas till intervallet $[-180^\circ, 180^\circ]$.

Focus Zoomar in övergångszonen i amplitudfunktionen. Förutsätter att kravspecifikation finns.

Radius Zoomar in i z-planet inom den radie som anges till höger.

Adjust Gain Justerar den konstanta förstärkningen. Anges i dB.

Övrigt

Muspekarens aktuella position i z-planet visas som radie och normerad frekvens (vinkel) vid **Radius** och **Frequency**.

Antal utlagda nollställena och poler visas vid **Zeros** respektive **Poles**.

Den konstanta förstärkningen som krävs för att $|H(f)|_{|f=0} = 1$ för lågpasfilter och $|H(f)|_{|f=0.5} = 1$ för högpasfilter, visas vid **Gain**.

Tips!

Poler kan endast placeras innanför enhetscirkeln. För att lägga en pol så nära enhetscirkeln som programmet tillåter, sikta strax utanför cirkeln.

Nollställena kan placeras på och innanför enhetscirkeln, om man bortser från reciproka nollställena. För att placera ett nollställe på enhetscirkeln, sikta strax utanför, så läggs det på enhetscirkeln.

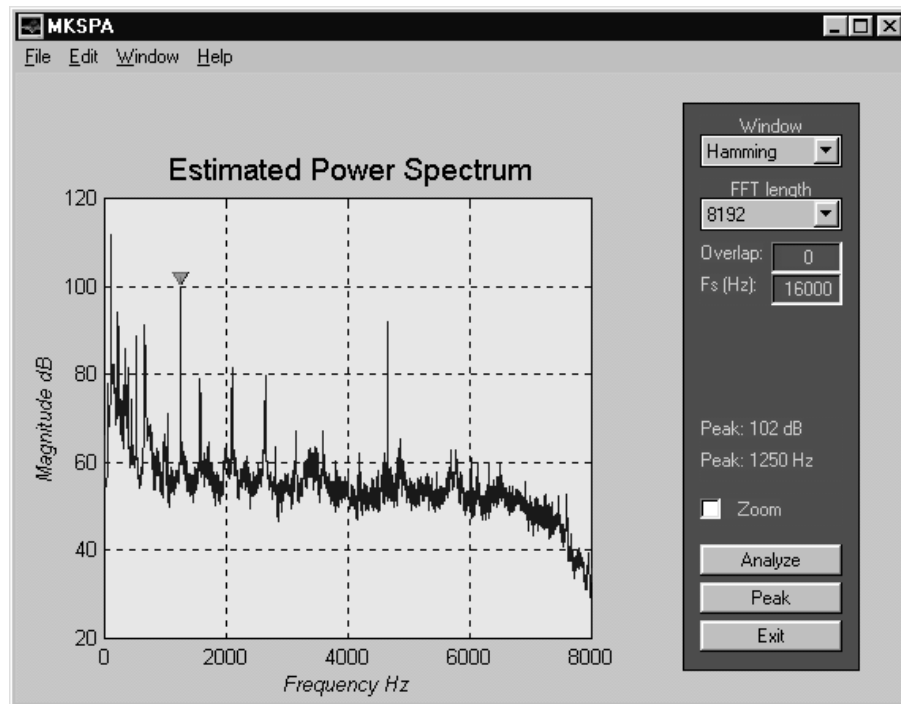
Om olika pollägen/nollställeslägen ska undersökas, använd funktionen `Replace last`, så ersätts senast angivna pol/nollställe då ett nytt läggs ut.

Om det skulle inträffa att ett fönster inte stängs, gör då så här: Peka på det fönster som ska stängas, och tryck vänster musknapp. Skriv därefter `delete(gcf)` vid prompten.

Spektralanalys, MKSPA

Beskrivning

MKSPA används för att analysera frekvensinnehållet i ljudfilerna. Programmet är uppbyggt kring MATLAB-funktionen `psd`, med tillägg för markering av toppar i spektrum.



Funktioner

Window Lista för val av fönsterfunktion.

FFT length Lista för val av FFT-längd.

Overlap Inmatning av hur många sampel segmenten ska överlappa varandra med.

Fs(Hz) Inmatning av samplingsfrekvens, som ligger till grund för graderingen av frekvensaxeln.

Zoom Anger om Matlabs zoom ska aktiveras.

Analyze Beräkning av spektrum, baserat på vald fönsterfunktion, FFT-längd, och antal överlappande sampel.

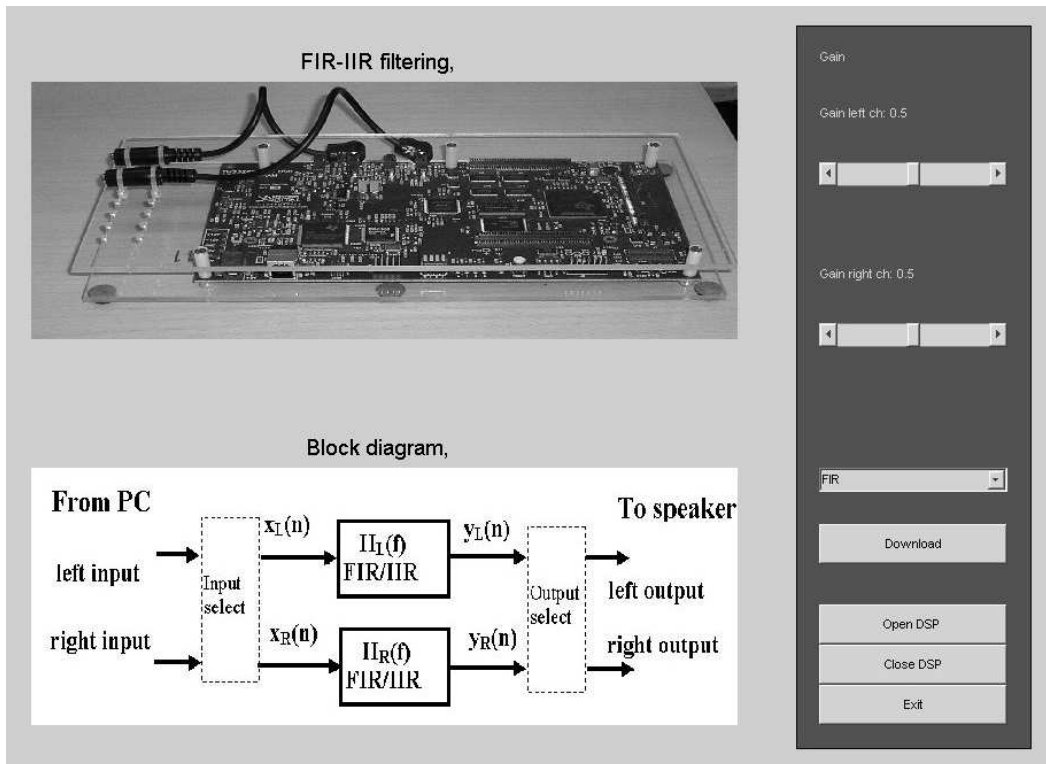
Peak Markerar toppar i spektrum. Genom att flytta muspekaren över grafen, markeras den största toppen i närheten av muspekaren. Amplitud (dB), samt frekvens (Hz) för aktuell topp visas vid textfälten **Peak:**. Den aktuella toppen markeras med en triangel.

Exit Avslutar programmet.

Filtrering i DSPn, filter_menu

Beskrivning

Programmet `filter_menu`:s funktioner beskrivs i korthet nedan.



Kommando:

`global cc B A, filter_menu`, `cc` är Matlab-address till DSPn.

Funktioner

Download Läser ner parametrar till DSPn. Filtret bestäms av $B(z)$ och $A(z)$.

Open DSP Startar CCS, öppnar Matlab-länk till DSPn, laddar ner och startar program i DSPn.

Close DSP Stänger DSP (Stänger Matlab-länken till DSPn. Programmet i DSP fortsätter tills nytt program laddas ner).

FIR Väljer FIR-filer.

IIR Väljer IIR-filer.

Exit Avslutar Matlab-programmet (Matlab-länken och programmet i DSPn påveckas ej).

11 Datorövning 7: DFT i teori och praktik

Denna datorövning syftar till att skapa förståelse för ett av de viktigaste matematiska verktygen inom signalbehandlingsområdet, den *Diskreta Fouriertransformen*, DFT.

En viktig och vacker tillämpning av DFT står att finna kommunikationssystem som använder en moduleringsmetod kallad *Orthogonal Frequency Division Multiplexing*, OFDM. Ett sådant system är ADSL². Andra tillämpningar där DFT och OFDM utnyttjas är i digital TV och i många nyare radionätverk.

En viktig egenskap hos DFT är att den kan beräknas med låg komplexitet, med hjälp av en metod som kallas FFT (*Fast Fourier Transform*). Observera att FFT endast är ett sätt att beräkna DFT på ett effektivt sätt, inte en ny transform. MATLAB använder FFT för att beräkna DFT, kommandot heter därför `fft`.

Tanken är att ni, när datorövningen är klar skall:

- Inse hur DFT representerar en diskret signal som dess koefficienter i en bas som utgörs av en uppsättning komplexa exponentialfunktioner.
- Förstå definitionen av DFT, och kunna använda den både till att dra slutsatser om egenskaper hos DFT transformen och till att beräkna DFT:n av signalen.
- Ha fått en inblick i sambanden mellan diskret och kontinuerligt spektrum och resultatet av en DFT beräkning. Speciellt skall MATLAB:s `fft` kommando kunna användas för att dra slutsatser om en signals frekvensinnehåll.
- Förstå begreppen cirkulär och linjär faltning och hur DFT:n kan utnyttjas för att beräkna dem. Du skall också ha förståelse för hur DFT kan utnyttjas i ett OFDM system.

11.1 DFT av sinus och cosinus

Vi beräknar först DFT av några sinussignaler och plotta upp absolutvärdet av DFT:n.

```
>> N=128;n=0:N-1;
>> x1=sin(2*pi*(10/N)*n);
>> x2=sin(2*pi*(10.5/N)*n);
>> X1=fft(x1); X2=fft(x2);
>> subplot(411);plot(x1);subplot(412);plot(abs(X1));
>> subplot(413);plot(x2);subplot(414);plot(abs(X2));
```

- Varför blir transformerna så olika för frekvenserna $\hat{f} = \frac{10}{128}$ och $\hat{f} = \frac{10.5}{128}$? Om du är osäker på skillnaden, plotta signalerna i tidsplanet och jämför.
- Gradera x-axeln $0 < \hat{f} < 1$.
- Plotta realdelen och imaginärdelen av `X1` (`real(X1)`, `imag(X1)`) istället. Vad händer? Vad händer om signalen är en cosinus istället för en sinus? Varför? (ledning: tänk på Eulers formler för cosinus och sinus)

11.2 DFT och faltning

Vi börjar med att skapa två sekvenser `x` och `y`.

```
>> x=[1 2 3 4 5 6 7 8 9];y=[1 5 3 2 7 9 8 6 4];
>> conv(x,y)
```

Vi beräknar nu samma faltning med hjälp av DFT:

²I ADSL används normalt inte begreppet OFDM utan istället *Discrete MultiTone*, DMT.

```
>> X=fft(x);Y=fft(y);XY=X.*Y;
>> real(ifft(XY))
```

Att vi använde `real` beror på att det blir små rester av imaginära tal kvar på grund av begränsad precision i de numeriska beräkningarna, dessa tas nu enkelt bort. När man skriver program själv kan det vara bra att kolla så att resultatet har mycket liten imaginärdel innan man tar bort den.

- Blir resultatet detsamma? Varför?

Cirkulär och linjär faltning

Cirkulär faltning:

$$y[n] = \sum_{m=0}^{N-1} h[m]x_{periodisk}[n-m] \quad (6)$$

Linjär faltning:

$$y[n] = \sum_{m=-\infty}^{m=\infty} h[m]x[n-m] \quad (7)$$

Som ni säkert vet gäller att sampling av en signal i tid leder till att dess spektrum blir periodiskt. Om man studerar definitionerna av DFT ser man att transformen och dess invers är identiska sånär som på ett konjugat (tecknet framför j) och en skalfaktor. Det är därför intuitivt att tänka sig att sampling i frekvens ger periodicitet i tid, det är ju i princip samma transform som används. DFT:n kan ses just som en sampling av spektrum och efter inverstransformering är signalen därför implicit periodisk. Detta ser vi aldrig direkt, eftersom vi normalt endast beräknar *en* period av inversen. Problem uppstår dock när faltning görs i frekvensplanet. Operationen motsvarar då faltning av de båda *periodiska* följderna. Man kan lätt övertyga sig om att detta inte ger samma resultat. Vi beräknar nu faltningen av samma signaler som ovan, fast en av dem konkatenerad med sig själv, för att utföra en sk cirkulär faltning, se formeln ovan.

```
>> x=[1 2 3 4 5 6 7 8 9];y=[1 5 3 2 7 9 8 6 4];
>> conv([x x],y)
```

- Känner du igen resultatet från den FFT baserade metoden ovan?
- Varför är resultatvektorerne olika långa i de båda fallen?
- Hur skall man gå tillväga för att åstakomma linjär faltning med FFT?

Ett sätt att utföra linjär faltning med hjälp av DFT är att utvidga signalen med nollor. Även om faltningen naturligtvis fortfarande är cyklisk så kommer den att ge samma resultat som en linjär faltning eftersom så många termer blir noll, rita upp detta!

Utvrida nu en av signalerna x eller y med lämpligt antal nollor och beräkna faltningen med hjälp av FFT som ovan.

- Fungerar det bättre nu?
- Hur många nollor behöver man lägga till om två signaler av längd N respektive M skall faltas?

Extrauppgift för den matematiskt intresserade

Kör följande MATLAB kod:

```
>> xx=imread('lena.tif');  
>> figure;colormap gray  
>> subplot(211);imagesc(xx);  
>> subplot(212);imagesc(real(fft(fft(xx))));
```

Vad händer? Det kan vara fövirrande att `fft` här appliceras på en matris. Resultatet blir helt enkelt en matris där kolonnerna är DFT:n av kolonnerna i den ursprungliga matrisen. Riktigt intressant blir det om man dessutom gör en radvis DFT, fast det tar vi en annan gång!

Vi har alltså konstaterat att "dubbel" DFT (nästan iallafall) innebär spegling i tidsplanet. Hur kan det komma sig? Använd definitionen av DFT direkt, och var noga med vilka index som används för följderna. Glöm heller inte den implicita periodiciteten vi behandlat tidigare. En annan liten parentes är att en konstant skalfaktor försvinner när vi använder `imagesc`, så om en sådan dyker upp i era beräkningar behöver ni inte ha gjort fel!

12 Extra övningsuppgifter

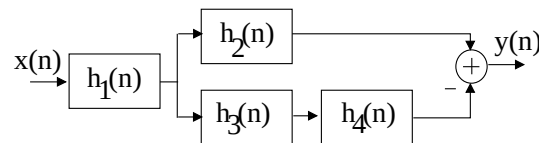
E1 För att visa effekten av vikning gör vi följande. Vi samplar in signalen $x(t) = 3 \cos(100\pi t) + 2 \sin(250\pi t)$ med sampeltakten $F_s = 200$ Hz utan att ha något analogt filter före samplingen. Vi lyssnar sedan på signalen med sampeltakten $F_s = 1000$ Hz (ideal rekonstruktion). Hur ser signalen ut efter rekonstruktionen? Matlab: Vid inspelning med ljudkort i PC filtreras den analoga signalen automatiskt med ett analogt filter med brytfrekvens $F_s/2$ före samplingen för att undvika vikningsdistor-sion. Vill vi tex illustrera exemplet ovan kan vi tex sampla med $F_s = 1000$ Hz vilket ställer gränsfrekvensen i ljudkortets filter till 500 Hz. Nu behåller vi bara vart femte sampel och har då reducerat sampeltakten till 200 Hz och detta motsvarar att vi samplat med 200 Hz men med det analoga filtret inställt på 500 Hz.

E2 Beräkna faltningen $x(n) * h(n)$ för nedanstående signaler.

- a) $x(n) = \{1111\}$, $h(n) = \{65432100\}$
- b) $x(n) = \{1111\}$, $h(n) = \{654\mathbf{3}2100\}$
- c) $x(n) = \{0001111\}$, $h(n) = \{11000\}$
- d) $x(n) = \{001111\}$, $h(n) = \{110\}$

MATLAB: Lös ovanstående faltningar i MATLAB (conv.m).

E3 Bestäm $h(n)$ för systemet nedan då $h_1(n) = \{0.5, 0.25, 0.5\}$, $h_2(n) = h_3(n) = (n + 1)u(n)$ och $h_4(n) = \delta(n-2)$. Bestäm utsignalen då $x(n) = \delta(n+2) + 3\delta(n-1) - 4\delta(n-3)$



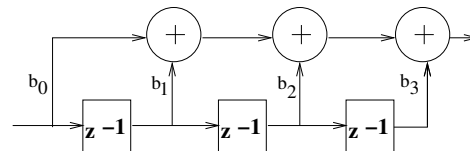
E4 Beräkna korrelationsfunktionen $r_{xx}(l)$ för följande sekvenser.

- a) $x(n) = \{1, 2, 1, 1\}$.
- b) $x(n) = \{1, 1, 2, 1\}$.

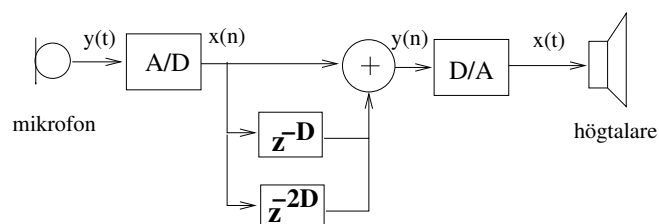
Vad är slutsatsen av a) och b) ovan?

MATLAB: Lös a) och b) ovan i MATLAB.

E5 Dimensionera nedanstående krets så att likspänningsförstärkningen blir 1 och så att frekvenserna $\omega = \pi/2$ och $\omega = \pi$ spärras. Bestäm också kretsens amplitudfunktion och fasfunktion.

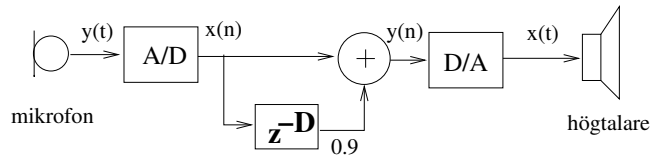


E6 Vi vill skapa ekoeffekt och använder oss av nedanstående koppling.



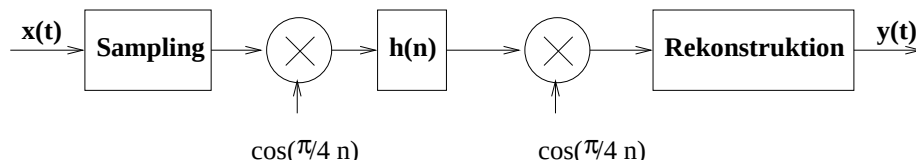
- Bestäm den digitala kretsens poler och nollställen för $D = 500$.
- Bestäm och skissa den digitala kretsens amplitudfunktion $|H(f)|$.

E7 Vi använder oss av nedanstående koppling för att få en ekoeffekt.



- Bestäm den digitala kretsens impulssvar.
- Bestäm den digitala kretsens amplitudfunktion $|H(f)|$.
- Bestäm den digitala kretsens poler och nollställen för $D=500$.

E8 Insignalen till nedanstående system är $x(t) = \cos(2\pi 1000t) + \cos(2\pi 6000t)$. Filtrets impulssvar $h(n) = u(n) - u(n - 8)$. Sampeltakten är 8 kHz.



Bestäm utsignalen $y(t)$ då rekonstruktionen är ideal.

MATLAB: Simulera systemet i MATLAB. Välj gärna ljud som insignal.

E9 Sekvensen $\{1, 1, 1, 1, 0, 0, 0, 0\}$, har DFT:n $X(k)$. Bestäm nu DFT:erna av sekvenserna

- $\{1, 0, 0, 0, 0, 1, 1, 1\}$.
- $\{0, 0, 1, 1, 1, 1, 0, 0\}$.

E10 Beräkna och skissa $|X(k)|$ där $X(k)$ är 8 punkters DFT av $x(n)$ med $x(n) = \{1, 1, 1, 0, 0, 0, 0, 1\}$

E11 Den signalbehandlingsintresserade eleven Hans hittar en stämgafl. Tyvärr framgår det ej vilken resonansfrekvens stämgafln har. Han kommer då på den lysande idén att utnyttja en spektrumanalysator. Spektrumanalysatorn är digital och gör en DFT av insignalen. Med spektrumanalysatorn inställd på 0 - 200 Hz (sampeltakt 400 Hz) avläser han en topp i spektrum vid frekvensen 138 Hz. Vilka värden på resonansfrekvensen kan stämgafln ha? Motivera svaret.

E12 Vi vill filtrera en uppmätt signal genom att bilda utsignalen $y(n)$ som medelvärde av de fem senaste insignalvärdena enligt

$$y(n) = \frac{1}{5} \sum x(k) = 0.2(x(n-4) + x(n-3) + x(n-2) + x(n-1) + x(n))$$

- Bestäm filtrets impulssvar $h(n)$.
- Bestäm filtrets amplitudfunktion $|H(f)|$.
- Skissa $|H(f)|$ för $0 \leq f \leq 1$.

E13 Utgående från signalen $x(n) = \sin(\pi/2 n)$ önskar man åstadkomma signalen $y(n) = \sin(\pi/2 n + \pi/3)$. Bestäm en krets som gör detta. Försök med ett FIR-filter.

13 Svar och lösningar till extra övningsuppgifter

E1

$$x_a(t) = 3\cos(2\pi \cdot 50t) + 2\sin(2\pi \cdot 125t)$$

$$F_s = \frac{1}{5 \cdot 10^{-3}} = 200Hz \qquad F_{rek} = \frac{1}{1 \cdot 10^{-3}} = 1000Hz$$

$$\begin{aligned} x(n) &= 3\cos\left(2\pi \frac{50}{200} n\right) + 2\sin\left(2\pi \frac{125}{200} n\right) \\ &= 3\cos\left(2\pi \frac{1}{4} n\right) + 2\sin\left(2\pi \frac{5}{8} n\right) = \\ &= 3\cos\left(2\pi \frac{1}{4} n\right) + 2\sin\left(-2\pi \frac{3}{8} n\right) \end{aligned}$$

$$F_{rek} = 1kHz \implies$$

$$\begin{aligned} y_a(t) &= 3\cos\left(2\pi \frac{1000}{4} t\right) + 2\sin\left(-2\pi \frac{3 \cdot 1000}{8} t\right) \\ &= 3\cos(2\pi \cdot 250t) - 2\sin(2\pi \cdot 375t) \end{aligned}$$

MATLABkod som illustrerar exemplet:

```
t=[0:.001:10];
x=3*sin(100*pi*t)+2*sin(250*pi*t);
y=x(1:5:length(x));
soundsc(x,1000);
soundsc(y,1000);
subplot(211),plot(t(1:100),x(1:100));
subplot(212),plot(t(1:100),y(1:100));
```

- E2**
- a) $y(n) = \{ \underline{6} \ 11 \ 15 \ 18 \ 14 \ 10 \ 6 \ 3 \ 1 \}$
 - b) $y(n) = \{ 6 \ 11 \ 15 \ \underline{18} \ 14 \ 10 \ 6 \ 3 \ 1 \}$
 - c) $y(n) = \{ 1 \ \underline{2} \ 2 \ 2 \ 1 \}$
 - d) $y(n) = \{ \underline{1} \ 2 \ 2 \ 2 \ 1 \}$

- E3**
- a) Parallell- och seriekoppling $\Rightarrow$

$$h(n) = h_1(n) * [h_2(n) - h_3(n) * h_4(n)]$$

b)

$$\begin{aligned} h_3(n) * h_4(n) &= (n-1)u(n-2) \\ h_2(n) - h_3(n) * h_4(n) &= \delta(n) + 2u(n-1) \\ h(n) &= \frac{1}{2}\delta(n) + \frac{5}{4}\delta(n-1) + 2\delta(n-2) + \frac{5}{2}u(n-3) \end{aligned}$$

c) Falta med en komponent av $x(n)$ i taget $\Rightarrow$

$h(n) * \delta(n+2)$	$\frac{1}{2}$	$\frac{5}{4}$	<u>2</u>	$\frac{5}{2}$	$\frac{5}{2}$	$\frac{5}{2}$	$\frac{5}{2}$	$\frac{5}{2}$	$\frac{5}{2}$	$\dots$
$h(n) * 3\delta(n-1)$	0	0	<u>0</u>	$\frac{3}{2}$	$\frac{15}{4}$	6	$\frac{15}{2}$	$\frac{15}{2}$	$\frac{15}{2}$	$\dots$
$h(n) * (-4\delta(n-3))$	0	0	<u>0</u>	0	0	-2	-5	-8	-10	-10
$\sum$	$\frac{1}{2}$	$\frac{5}{4}$	<u>2</u>	4	$\frac{25}{4}$	$\frac{13}{2}$	5	2	0	0
Utsignalen blir $y(n) = \{\frac{1}{2} \ \frac{5}{4} \ \underline{2} \ 4 \ \frac{25}{4} \ \frac{13}{2} \ 5 \ 2\}$										

E4 a)

$$\begin{aligned} x(n) &= \{1 \ 2 \ 1 \ 1\} \\ e_{xx}(\ell) &= \sum x(n)x(n-\ell) = \{1 \ 3 \ 5 \ 7 \ 5 \ 3 \ 1\} \end{aligned}$$

b)

$$\begin{aligned} y(n) &= \{1 \ 1 \ 2 \ 1\} \\ r_{yy}(\ell) &= \{1 \ 3 \ 5 \ 7 \ 5 \ 3 \ 1\} \end{aligned}$$

E5 Välj nollställe $z = j$, $z = -j$, $z = -1$. Detta ger $H(z) = b_0(1 + z^{-1} + z^{-2} + z^{-3})$.

DC-nivå =1 ger $b_0 = 1/4$ och $b_0 = b_1 = b_2 = b_3 = 0.25$.

Och $|H(\omega)| = 1/4 \left| \frac{\sin(2\omega)}{\sin(\omega/2)} \right| = 1/2 |\cos(3\omega/2) + \cos(\omega/2)|$,

$\arg(H(\omega)) = -3\omega/2 + \pi$ (om $\pi/2 < \omega < \pi$) (Och $H(\omega)$ periodisk).

E6 $H(z) = 1 + z^{-D} + z^{-2D} = \frac{z^{2D} + z^D + 1}{z^{2D}}$ $|H(\omega)| = |1 + e^{-j\omega D} + e^{-j\omega 2D}| = |1 + 2\cos\omega D|$.
 poler: $D = 1000$ i origo, nollställen: $z^{2D} + z^D + 1 = 0$, ger $z^D = 0.5(-1 \pm j\sqrt{3}) = e^{j\pm\frac{2\pi}{3}}$ $e^{j2\pi k}$, $k = 0, 1, \dots, D-1$, och slutligen $z_k = e^{\pm j\frac{2\pi}{3D}}$ $e^{j2\pi/D k}$

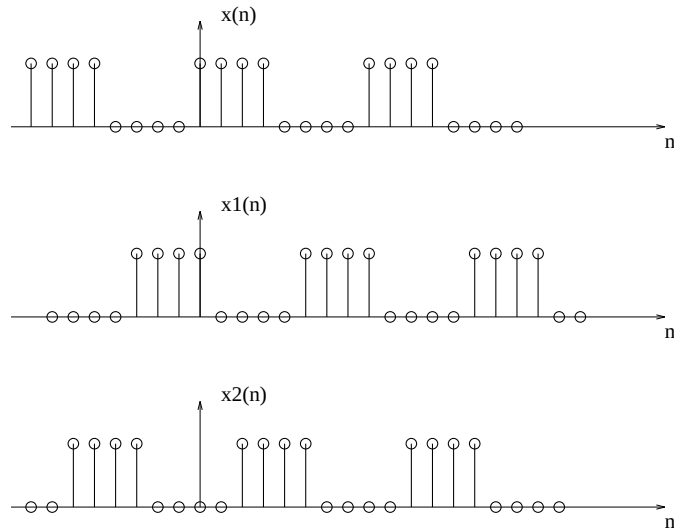
E7 a) $y(n) = x(n) + 0.9x(n-D)$, $h(n) = \delta(n) + 0.9\delta(n-D)$

b) $H(z) = 1 + 0.9Z^{-D} = \frac{z^D + 0.9}{z^D}$, $D = 500$, poler 500 i origo, nollställen $z^{500} = -0.9 = 0.9e^{j2\pi k + j\pi}$, $z_k = 0.9^{1/500}e^{j2\pi k/500 + j\pi/500}$, $k = 0, 1, 2, \dots, 499$ (ligger på en cirkel).

E8 $|H(f)| = \left| \frac{\sin 4\omega}{\sin \omega/2} \right|$, mult med $\cos(2\pi/8 n)$ flyttar spektrum $\pm 1/8$, $H(f)$ spärrar alla frekvenser utom $f = 0$ (rita spektra). Ger $y(t) = 4\cos(2\pi 1000t)$

E9 $X(k)$ $k = 0 \dots 7$ given.

DFT periodiserar $x(n)$



$$\left. \begin{aligned} x_1(n) &= x((n-5))_8 \\ x_2(n) &= x((n-2))_8 \end{aligned} \right\} \Rightarrow$$

$$X_1(k) = X(k) e^{-j2\pi \frac{k}{8} 5}$$

$$X_2(k) = X(k) e^{-j2\pi \frac{k}{8} 2}$$

E10 Låt $x_1(n) = \{1, 1, 1, 1, 0, 0, 0, 0\}$. $x(n)$ är cirkulärt skift av $x_1(n)$, skift påverkar endast fasen. Således $|X(k)| = |X_1(k)| = \left| \frac{\sin 2\pi \frac{k}{4}}{\sin 2\pi \frac{k}{16}} \right|$

E11 $f_0 = \pm 138 + n \cdot 400 = \{138, 262, \dots\}$.

E12

a) $h(n) = 1/5 \{1, 1, 1, 1, 1\} = 1/5(\delta(n) + \delta(n-1) + \delta(n-2) + \delta(n-3) + \delta(n-4))$

b) $H(z) = 1/5 \sum_{n=0}^4 z^{-n} = 0.2 \frac{1-z^{-5}}{1-z^{-1}}$

c) $H(f)$ periodisk sinc, $|H(f)| = 0$ för $f = 0.2, 0.4, 0.6, 0.8$

E13 Prova med 1:a ordn, $H(z) = b_0 + b_1 z^{-1}$. $H(\omega)|_{\omega=\pi/2} = e^{j\pi/3}$ ger $b_0 = 1/2$, $b_1 = -\sqrt{3}/2$